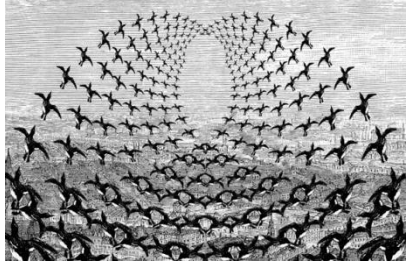


Crowd simulation

Taku Komura

Animating Crowds

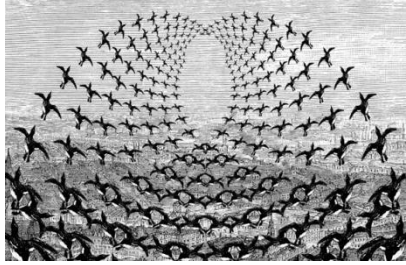


- We have been going through methods to simulate individual characters
- What if we want to simulate the movement of crowds?
  - Pedestrians in the streets
  - Flock of birds, school of fishes
  - People in panic



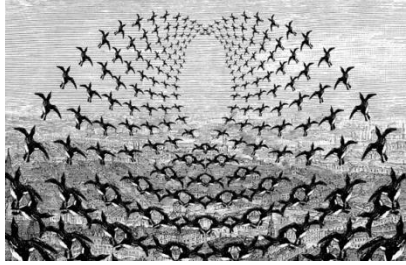


Why need a crowd simulation ?

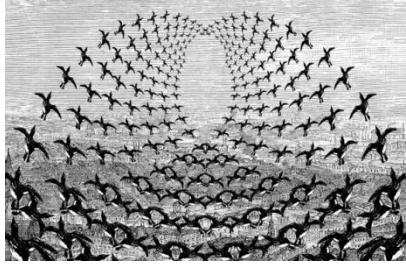


- Real scenes with many people
  - Expensive to hire many extra actors
- Computer games
  - Want the characters to be interacting/responding to the user inputs
- Safety reasons:
  - Need to evaluate the safety of buildings
- Autonomous driving
  - Generating training data for the system

Important Factors



- Autonomous
- Realistic
  - Paths
  - Interactions
  - Series of actions
- Fast
- Controllable
- Centralized vs. Decentralized

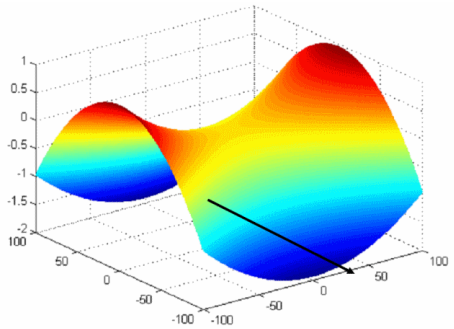


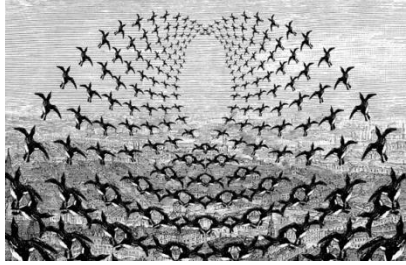
Overview

- Classic Agent-based Methods
- Flocking
- Adding intelligence
  - Intention generator
  - Autonomous pedestrians
- Patch-based methods
  - Create scenes by building blocks
- Collision avoidance
  - Mathematical models
  - Global methods (Continuum crowds)







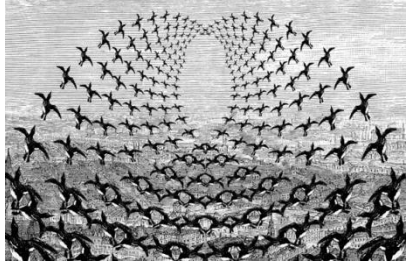


Flocking Models – Most basic agent model (Reynolds '87)

- The agents interact based on simple dynamics
- Good to simulate
  - Flock of birds flying,
  - School of fishes swimming

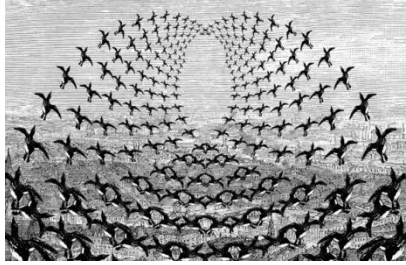


Forces applied to individuals



- Separation
- Alignment
- Cohesion
- Avoidance

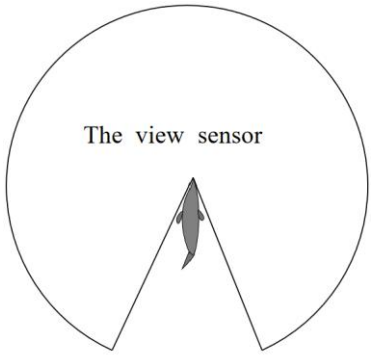
Separation

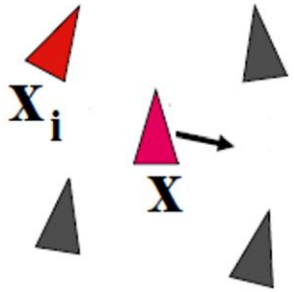


- Try to avoid running into local flock-mates
- Use a perception volume to limit visible flock-mates

$$\mathbf{F_s} = \sum_{i \in S} \left(\frac{\mathbf{x} - \mathbf{x_i}}{\|\mathbf{x} - \mathbf{x_i}\|^2} \right)$$

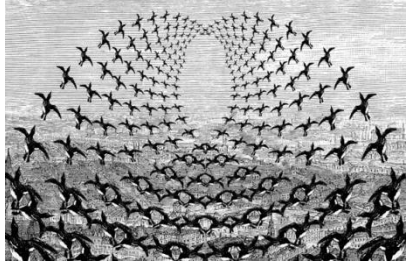
S: surrounding entities





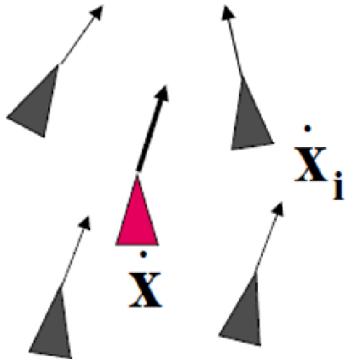
Separation: Fly away from entities that are too close

Alignment



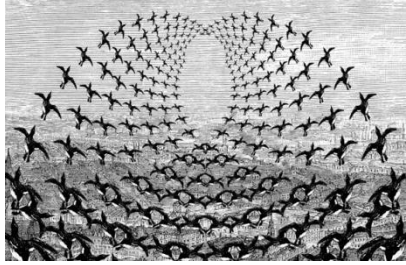
- Try to fly in the same direction as local flock-mates
- Makes every one fly in the same direction

$$\mathbf{F_A} = \sum_{i \in S} (\dot{\mathbf{x}}_\mathbf{i} - \dot{\mathbf{x}})$$



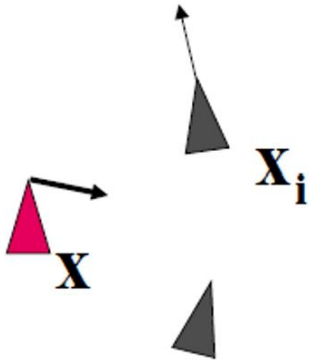
Alignment: Steer towards average velocity

Cohesion



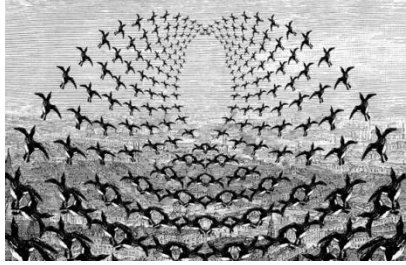
- Try to move toward the average location of the local flock-mates
  - Spaces everyone out evenly
  - Keep boundary members toward the group

$$\mathbf{F_c} = \sum_{i \in S}^{N} \frac{\mathbf{x_i}}{N} - \mathbf{x}$$



Cohesion: Steer towards average position

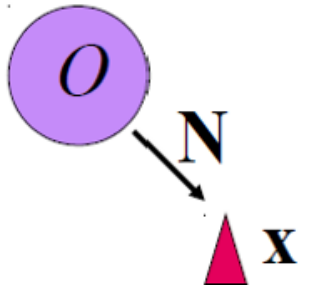
Avoidance



- Try to avoid obstacles
  - The closer the character is, the stronger the force

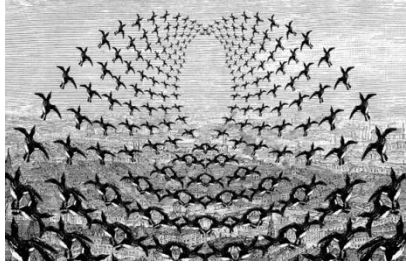
$$\mathbf{F_O} = \frac{\boldsymbol{N}}{D(O, \mathbf{x})}$$

  - N: the normal vector of the obstacle
  - D(O,x) : distance to the obstacle at the point that is closest to x



Avoidance: Steer away from the obstacle

Computing the acceleration



The forces are added to compute the acceleration of the agent

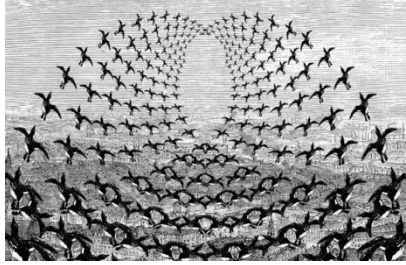
$$Ma = w_S F_S + w_C F_C + w_A F_A + w_O F_O$$

Then update the velocity and the position by integration

$$v' = v + a\Delta t$$

$$x' = x + v\Delta t$$

Combining Commands



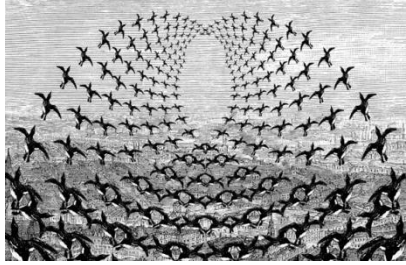
- The forces are combined through the weights

$$w_S, w_C, w_A, w_O$$

  - Ex. High for avoidance, low for cohesion

Simply summing the weighted sum may cause issues

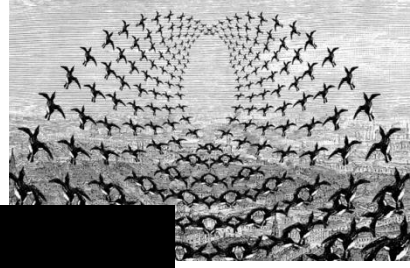
Combining Commands



- Option: Apply in order of highest weight, until a max (absolute sum of) acceleration is reached
  - Ensures that high priority things happen
  - $w_O \gg w_S, w_C, w_A,$

https://eater.net/boids

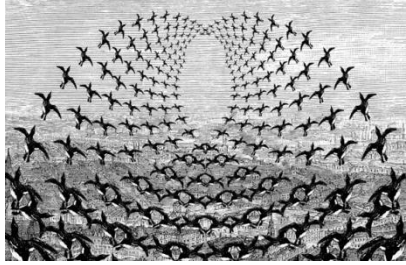
http://www.youtube.com/watch?v=4kPSDW7gQHA



COURSE: 07
COURSE ORGANIZER: DEMETRI TERZOPOULOS

"BOIDS DEMOS"
CRAIG REYNOLDS
SILICON STUDIOS, MS 3L-980
2011 NORTH SHORELINE BLVD.
MOUNTAIN VIEW, CA 94039-7311

Another application of flocking: Simulating dynamical features of escape panic : Helbing '00



- A model to simulate the crowd under panic

$$m_i \frac{d\vec{v}_i}{dt} = m_i \frac{\vec{v}_i^{\,o}(t)\vec{e}_i^{\,o}(t) - \vec{v}_i(t)}{\tau_i} + \sum_{j \neq i} \vec{f}_{ij} + \sum \vec{f}_{iw}$$

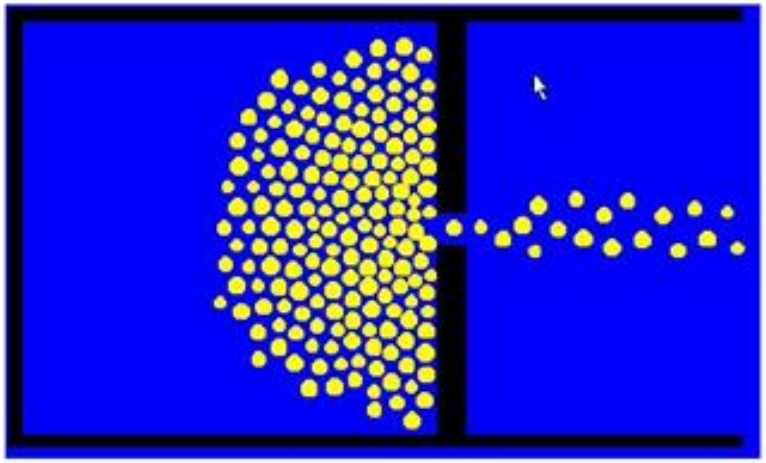
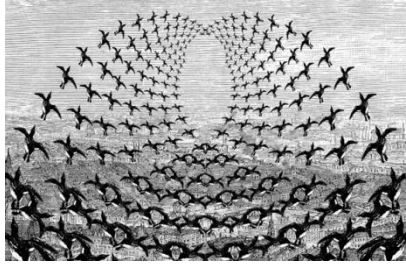


Figure 1: An image of Helbing´s Model Simulation.

http://www.youtube.com/watch?v=mUKYtOXg5Zw

Simulating dynamical features of escape panic : Helbing ’00



- f_{ij}, the force to stay away from other entities,
- f_{iw} the force to stay away from walls
- v_i: velocity of entity *i*
- $v_i^o e_i^o$: the desired velocity of entity *i*

$$m_i \frac{d\vec{v}_i}{dt} = m_i \frac{\vec{v}_i^{\,o}(t)\vec{e}_i^{\,o}(t) - \vec{v}_i(t)}{\tau_i} + \sum_{j \neq i} \vec{f}_{ij} + \sum \vec{f}_{iw}$$

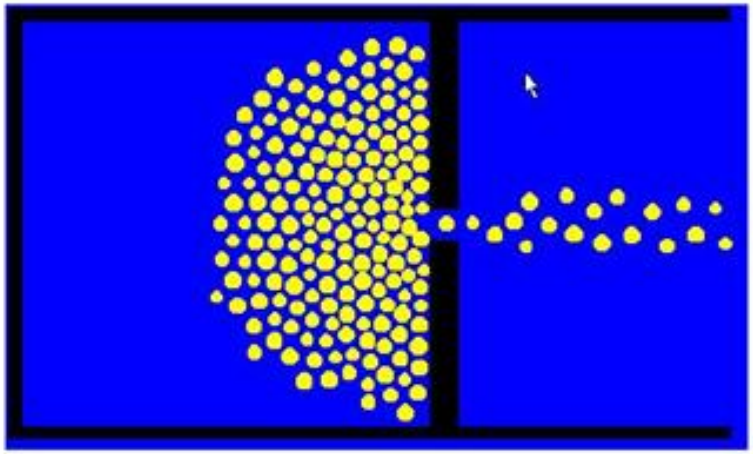
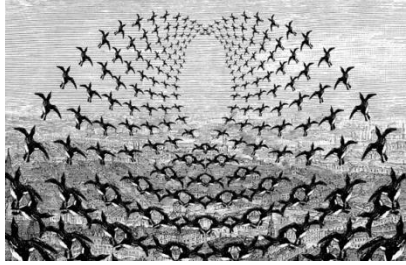


Figure 1: An image of Helbing´s Model Simulation.

Simulating dynamical features of escape panic : Helbing '00



- f_{ij}, the force to stay away from other entities,
- f_{iw} the force to stay away from walls
- v_i : velocity of entity *i*
- $v_i^o e_i^o$: the desired velocity of entity *i*

$$m_i \frac{d\vec{v}_i}{dt} = m_i \frac{\vec{v}_i^{\,o}(t)\vec{e}_i^{\,o}(t) - \vec{v}_i(t)}{\tau_i} + \sum_{j \neq i} \vec{f}_{ij} + \sum \vec{f}_{iw}$$

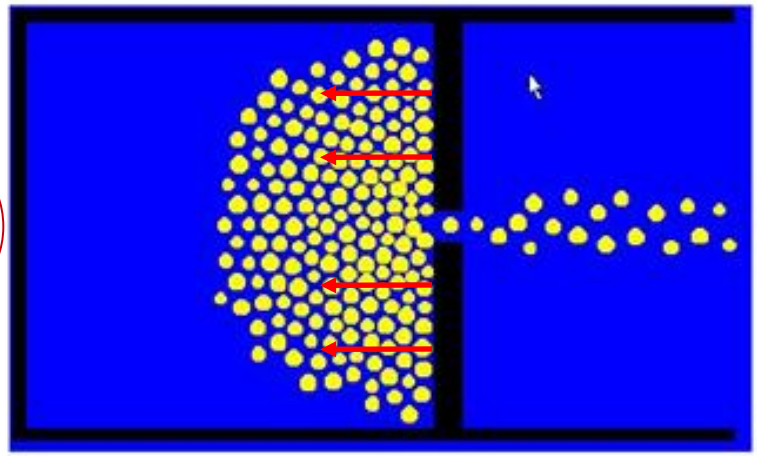
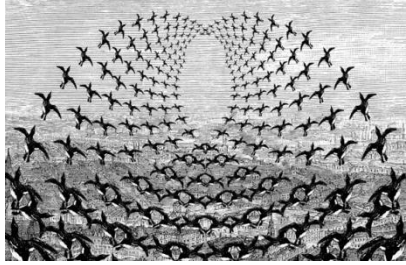


Figure 1: An image of Helbing´s Model Simulation.

Simulating dynamical features of escape panic : Helbing '00



- f_{ij}, the force to stay away from other entities,
- f_{iw} the force to stay away from walls
- v_i: velocity of entity *i*
- $v_i^o e_i^o$: the desired velocity of entity *i*

$$m_i \frac{d\vec{v}_i}{dt} = m_i \frac{\vec{v}_i^{\,o}(t)\vec{e}_i^{\,o}(t) - \vec{v}_i(t)}{\tau_i} + \sum_{j \neq i} \vec{f}_{ij} + \sum \vec{f}_{iw}$$

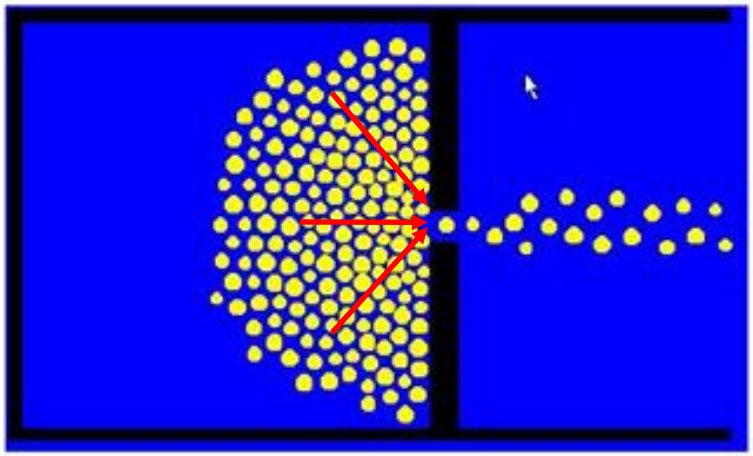
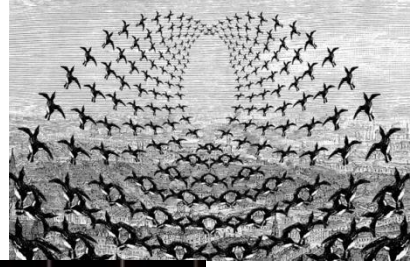
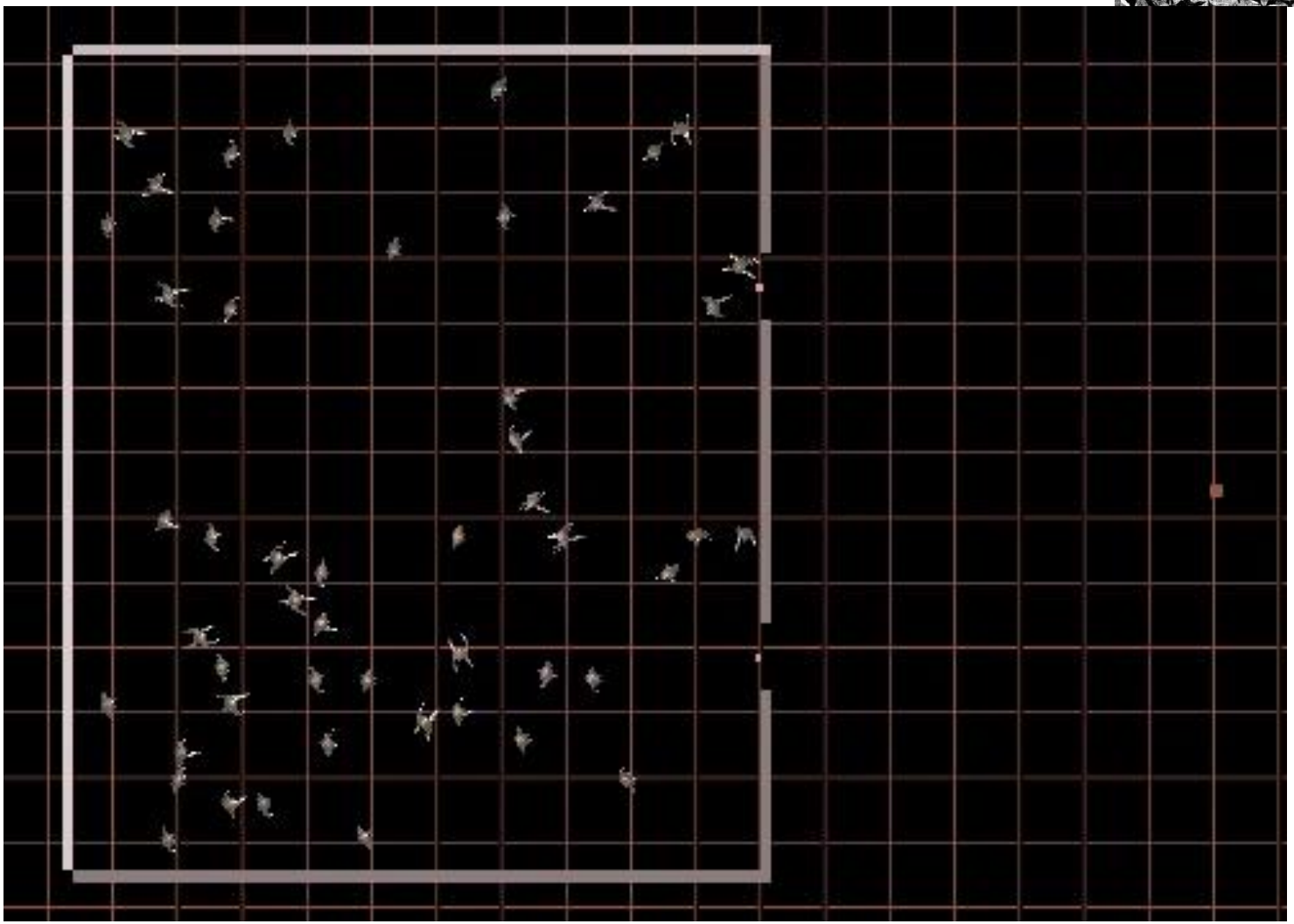
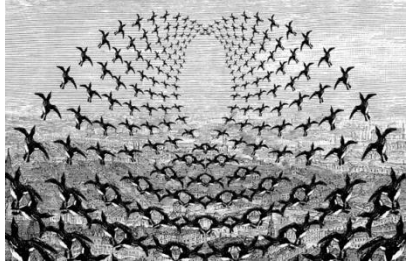


Figure 1: An image of Helbing´s Model Simulation.

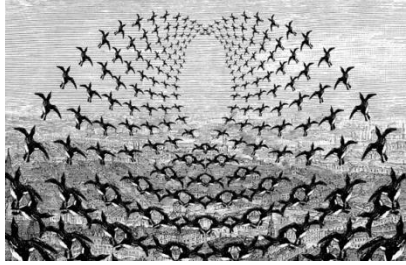




Flocking Evaluation



- Advantages:
  - Complex behaviour and phenomena from simple rules
  - Many types of behaviour can be expressed with different rules and parameters
- Disadvantages:
  - Can be difficult to set parameters to achieve desired result
  - Problems regarding strength of forces

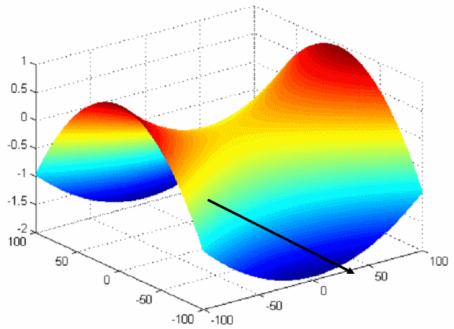


Overview

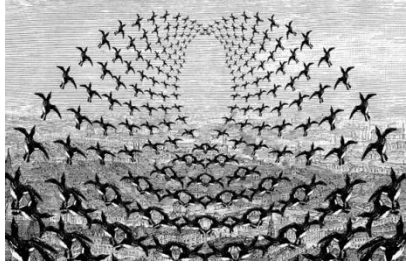
- Classic Agent-based Methods
- Flocking
- **Adding intelligence**
  - Intention generator
  - Autonomous pedestrians
- Patch-based methods
  - Create scenes by building blocks
- Collision avoidance
  - Mathematical models
  - Global methods (Continuum crowds)





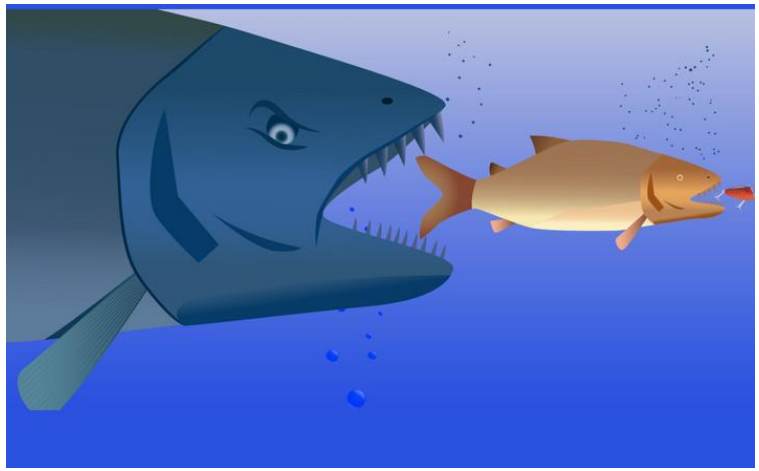


Making the Agents Intelligent

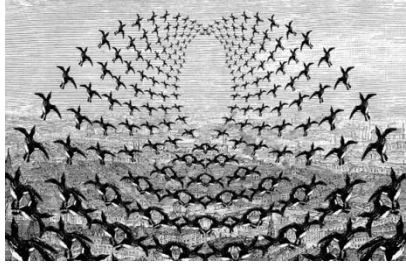


- Let's think of fish
- How can we make fish appear intelligent?



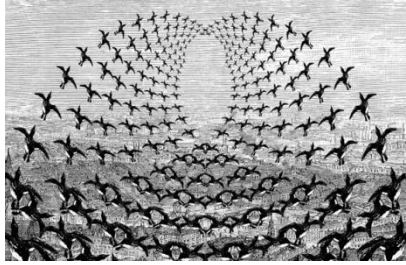


Making the Agents Intelligent



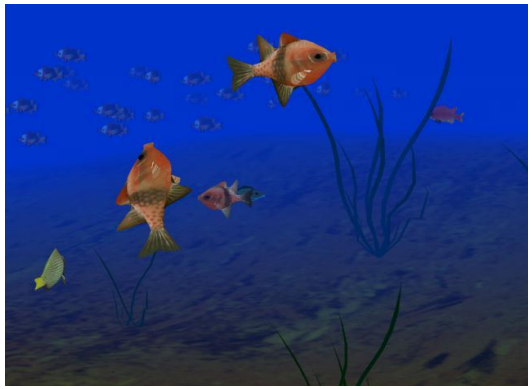
- Designing the details of the behaviours
  - What to do when you see a food?
  - What to do when you see a predator?
- Intention generator
  - A controller that decides the actions of the agents

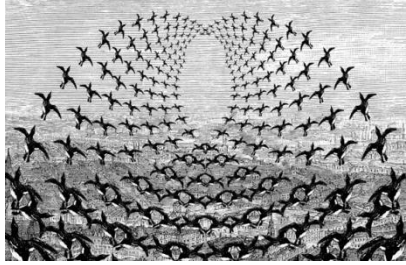
Artificial Fish
-Terzopoulos et al. SIGGRAPH ‘94



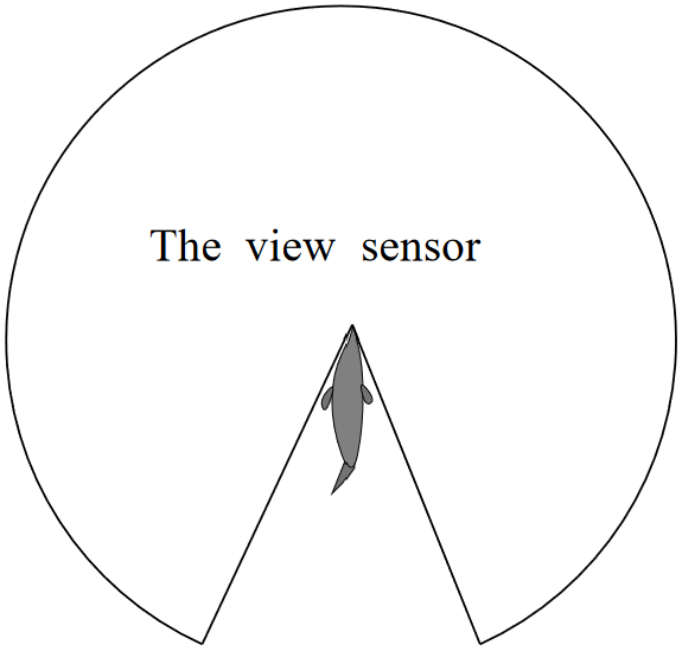
- Given sensory perception such as
  - vision
- and internal state variables quantifying
  - Hunger
  - Libido
  - Fear

the motor output is decided.



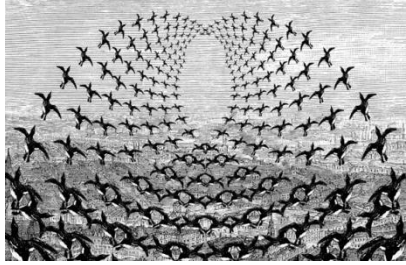


Perception Volume



Simulating the sight of the entity

Intention Generator



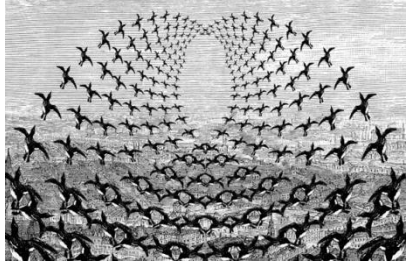
1. First check the sensory information for collision
2. If any close predator, either
   - School
   - Escape

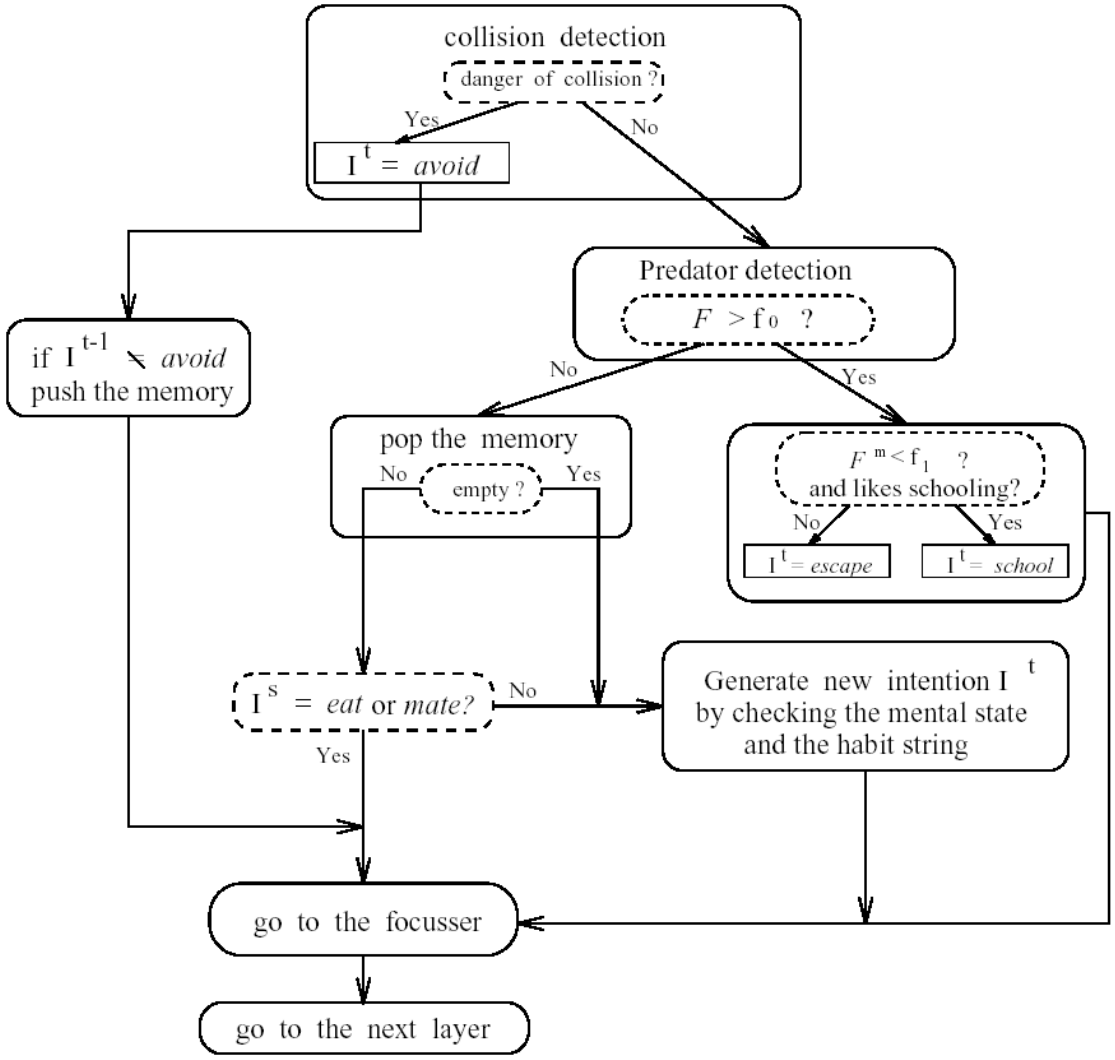
Otherwise if hungry eat

If full mate

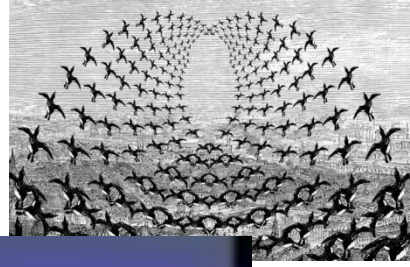
http://www.youtube.com/watch?v=RHt_8ZYQVZw

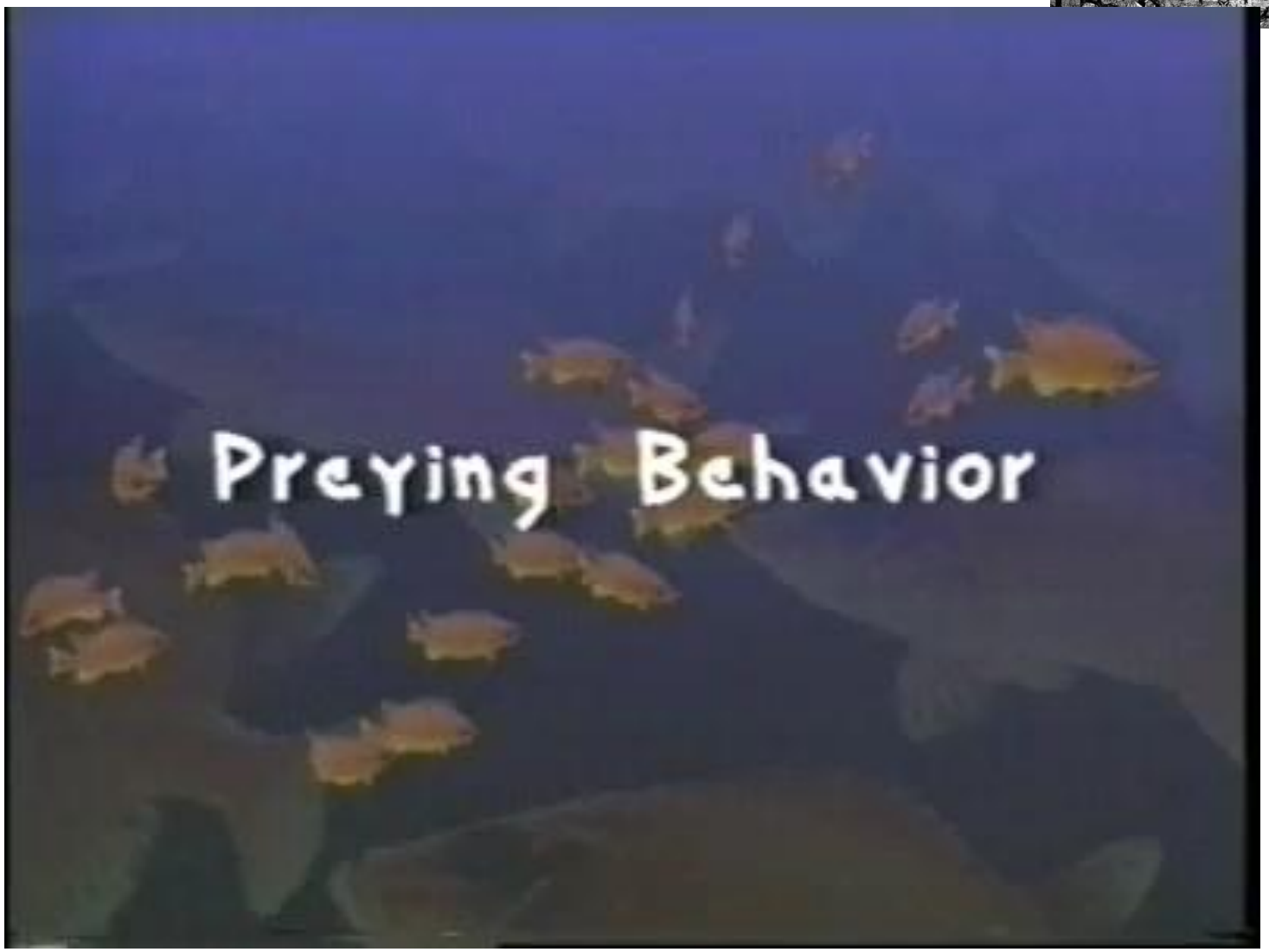
Intention Generator



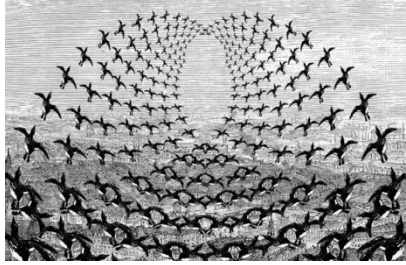


- Decision making for the fish

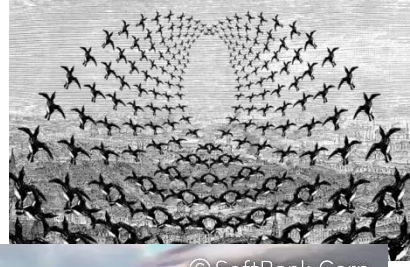



Preying Behavior

Applications



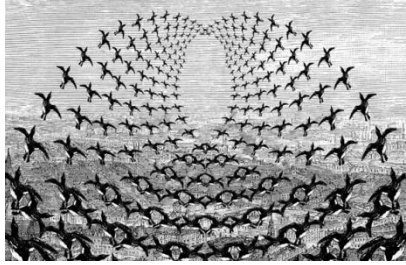
1. Computer animation
2. Artificial life
3. Preparing training data for fish monitoring





Watatumi:
an AI-powered smart fish farm software suite

We introduce Watatuni, an AI-powered smart fish farm software suite.

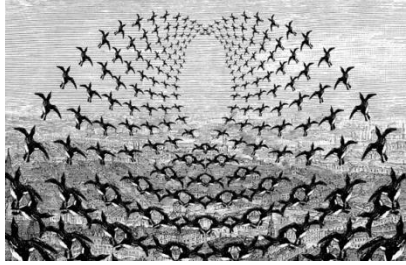


What about humans?

- Can we do something similar?
- But humans are a bit more complex
  - Not only eating or escaping
  - Have destinations
  - The motions are more complex
    - Biped locomotion
  - Socializing
    - Grouping,
    - Talking to somebody you know

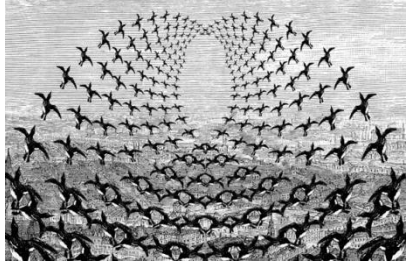
A more complex model is needed

Autonomous Pedestrians Shao et al. 2005

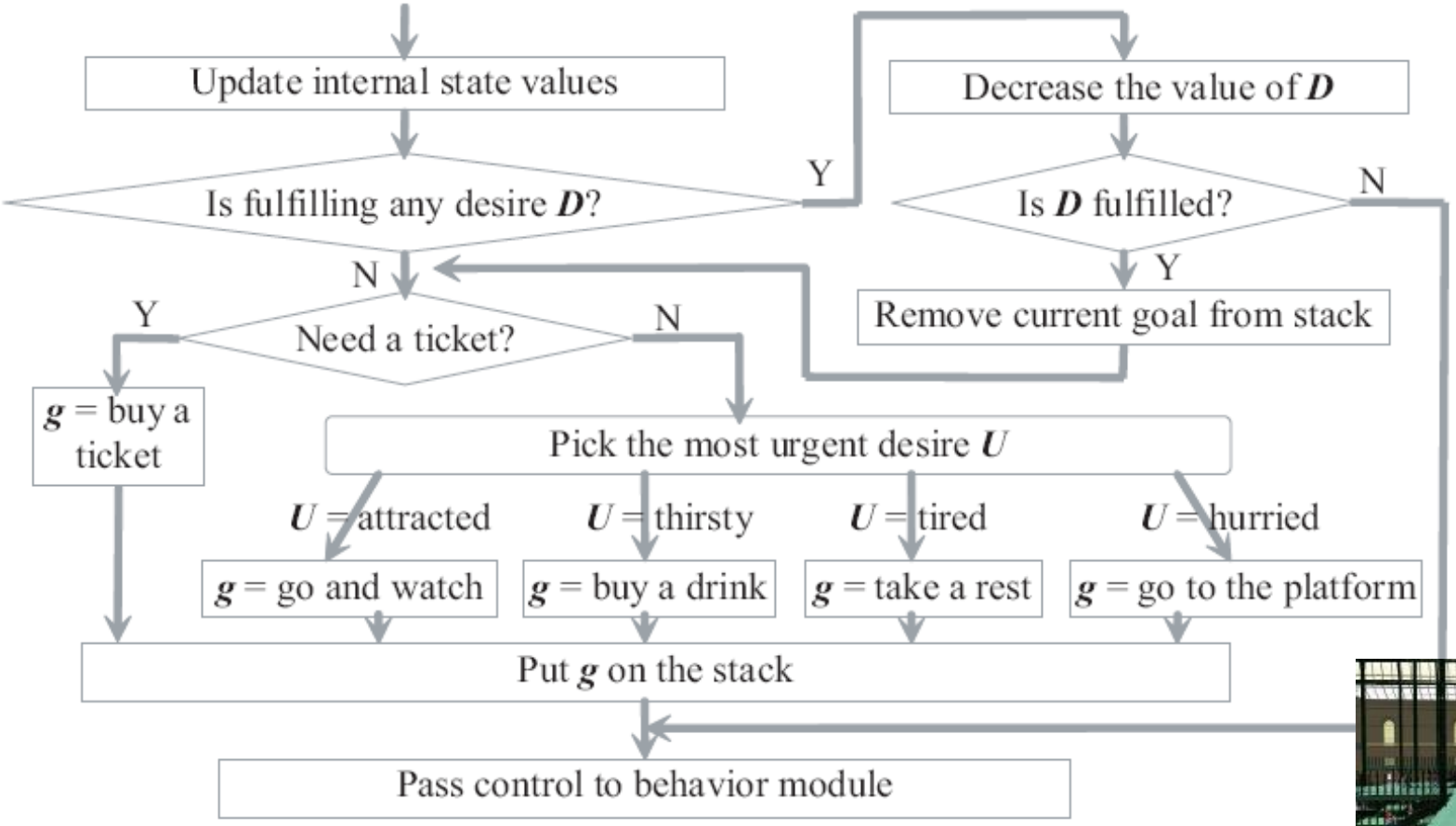


- A more complex intention generator
- Collision avoidance with other objects, other pedestrians
- Path planning to reach a goal – using A* search
- Kinematic motion synthesis
  - Locomotion, queuing, drinking, sitting, dancing etc.

Behavior model : Autonomous Pedestrians

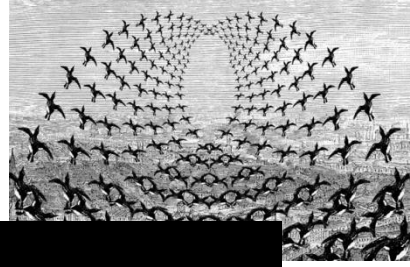


- Intention generator: More complex than a fish





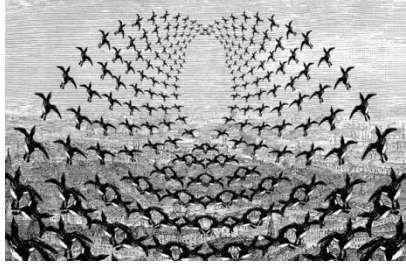
http://www.youtube.com/watch?v=cqG7ADSvQ5o



Following an Autonomous Pedestrian

(this video has sound)

Cognitive State



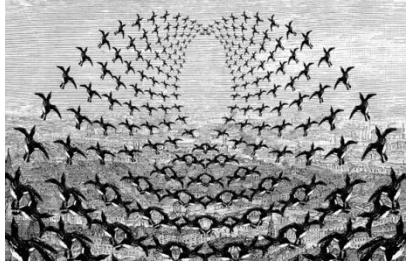
Internal parameters representing the

- tiredness
- thirst
- curiosity
- need for a ticket

are prepared.

When the value of mental state variable reaches a threshold, an action is activated.

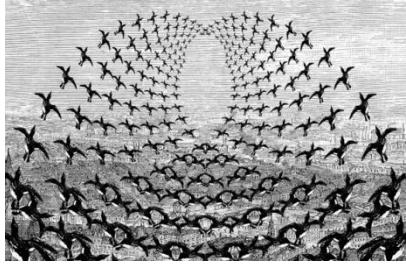
Autonomous Pedestrians: Summary



A hand-tuned controller and a series of modules prepared to synthesize complex human behaviors

The controller will grow as the number of motions/interactions increase

Is there a way to decentralize such a controller?

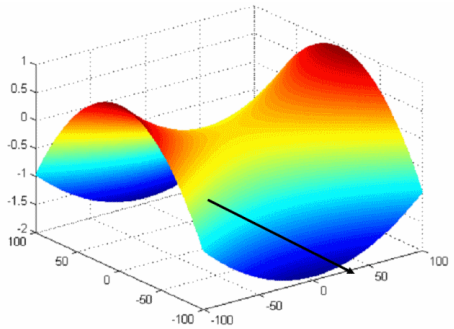


Overview

- Classic Agent-based Methods
- Flocking
  - Intention generator
  - Autonomous pedestrians
- **Patch-based methods**
  - Create scenes by building blocks
- Collision avoidance
  - Mathematical models
  - Global methods (Continuum crowds)

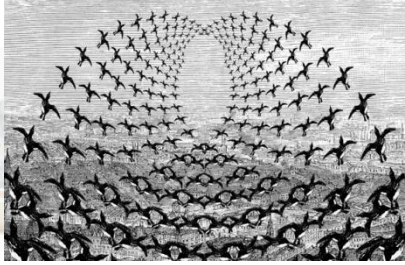






Interactions with the Environment





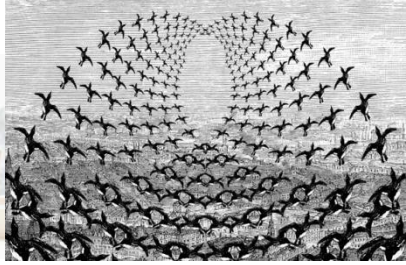




- Examples
  - At bus stops, people stop and queue
  - At an elevator, people wait for it and ride on it when it comes and travel to different floors
  - TV in the living room : turn on the TV and sit on the sofa to watch TV

Interactions with the Environment





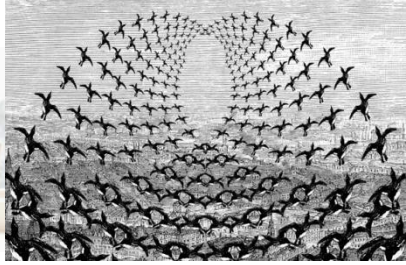




- Examples
  - At bus stops, people stop and queue
  - At an elevator, people wait for it and ride on it when it comes and travel to different floors
  - TV in the living room : turn on the TV and sit on the sofa to watch TV
- **Preparing a controller to interact with all these objects is not so easy**

Interactions with the Environment



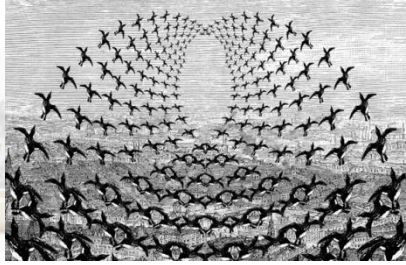






- Examples
  - At bus stops, people stop and queue
  - At an elevator, people wait for it and ride on it when it comes and travel to different floors
  - TV in the living room : turn on the TV and sit on the sofa to watch TV
- Preparing a controller to interact with all these objects is not so easy
- **Need various object / motion data**

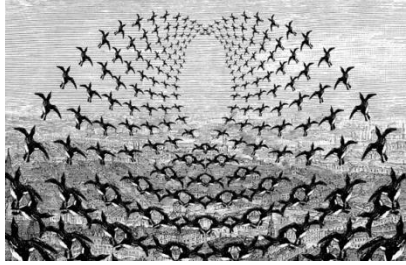
Interactions with the Environment



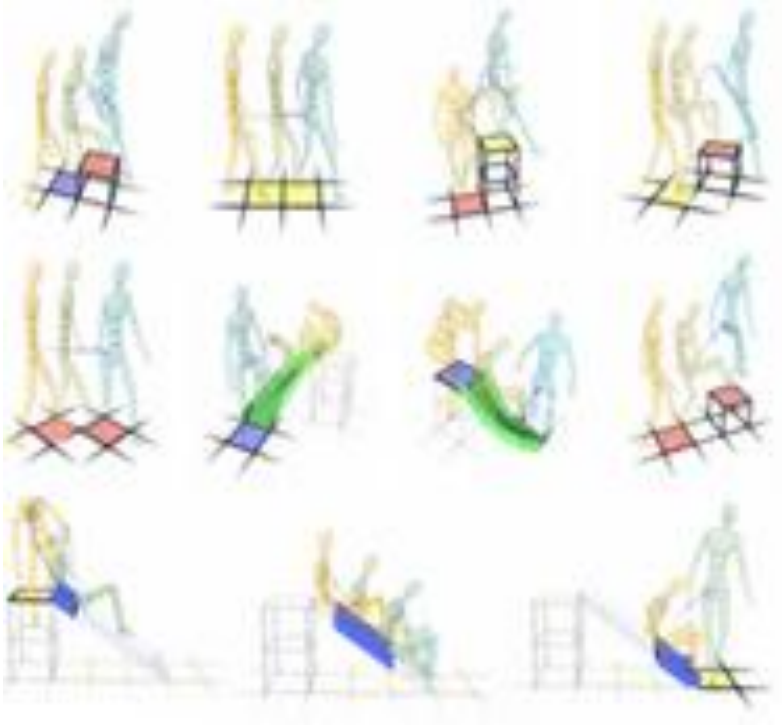
- Examples
  - At bus stops, people stop and queue
  - At an elevator, people wait for it and ride on it when it comes and travel to different floors
  - TV in the living room : turn on the TV and sit on the sofa to watch TV
- Preparing a controller to interact with all these objects is not so easy
- Need various motion/object data

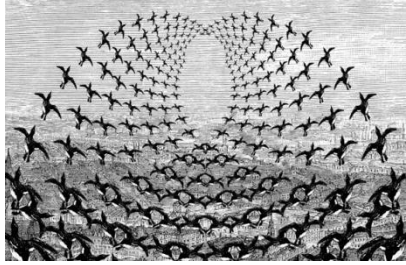
- **IDEA:** associate such motions with the objects
  - Once the character comes across such objects, they launch the associated motion saved with the object



Embedding motions into the environment : Advantages of the approach

- Efficient data handling
  - Each agent holds the data only needed at that moment
- The control is decentralized
  - The system is scalable, and large crowds can be simulated





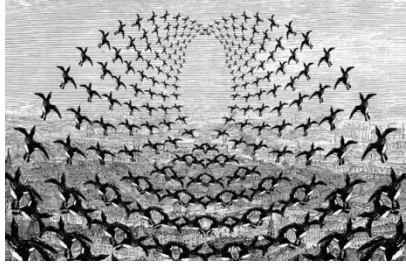
Patch-based approaches

- Pre-compute the patches (building blocks) which include the characters and the environment
- Concatenate them to generate large scale scenes during runtime
  - Motion Patches
  - Crowd patches





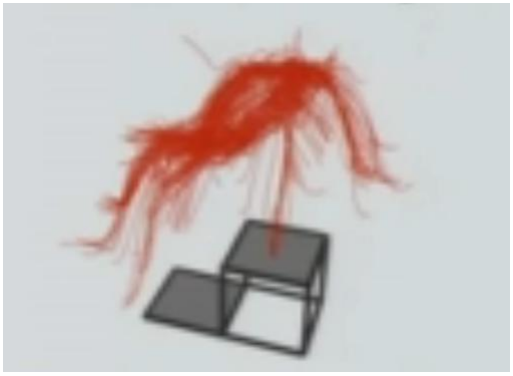
Motion Patches



Motion Patch = Shape + Motion

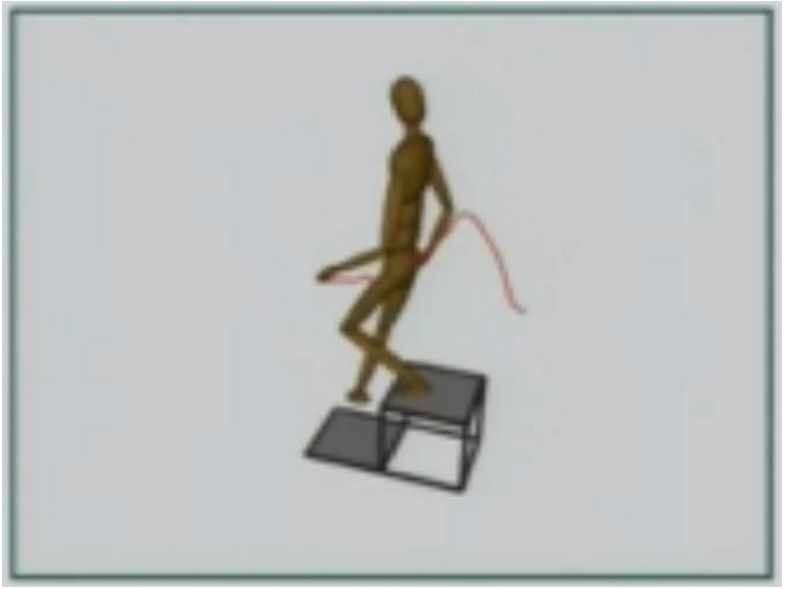
Motion

A set of motion segments

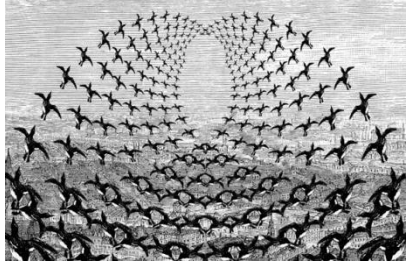


Shape

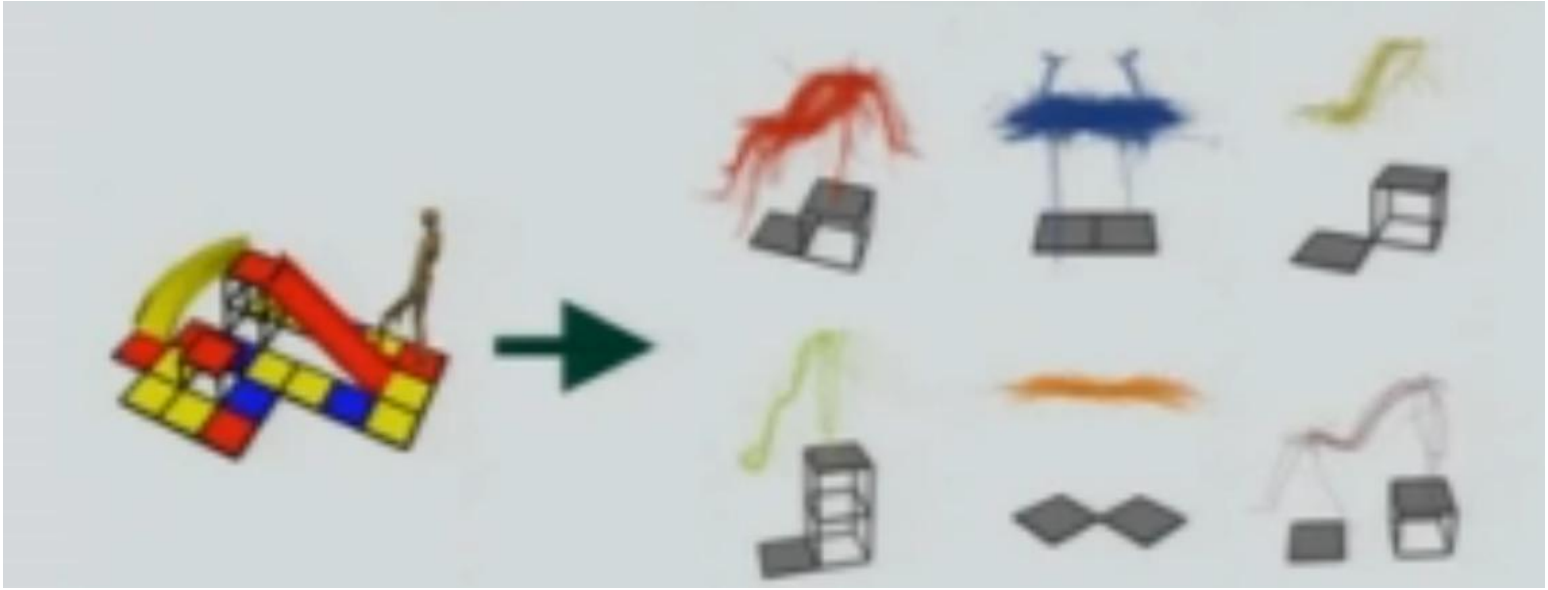
A set of unit objects

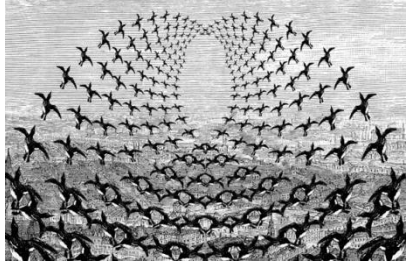


Generating Motion Patches



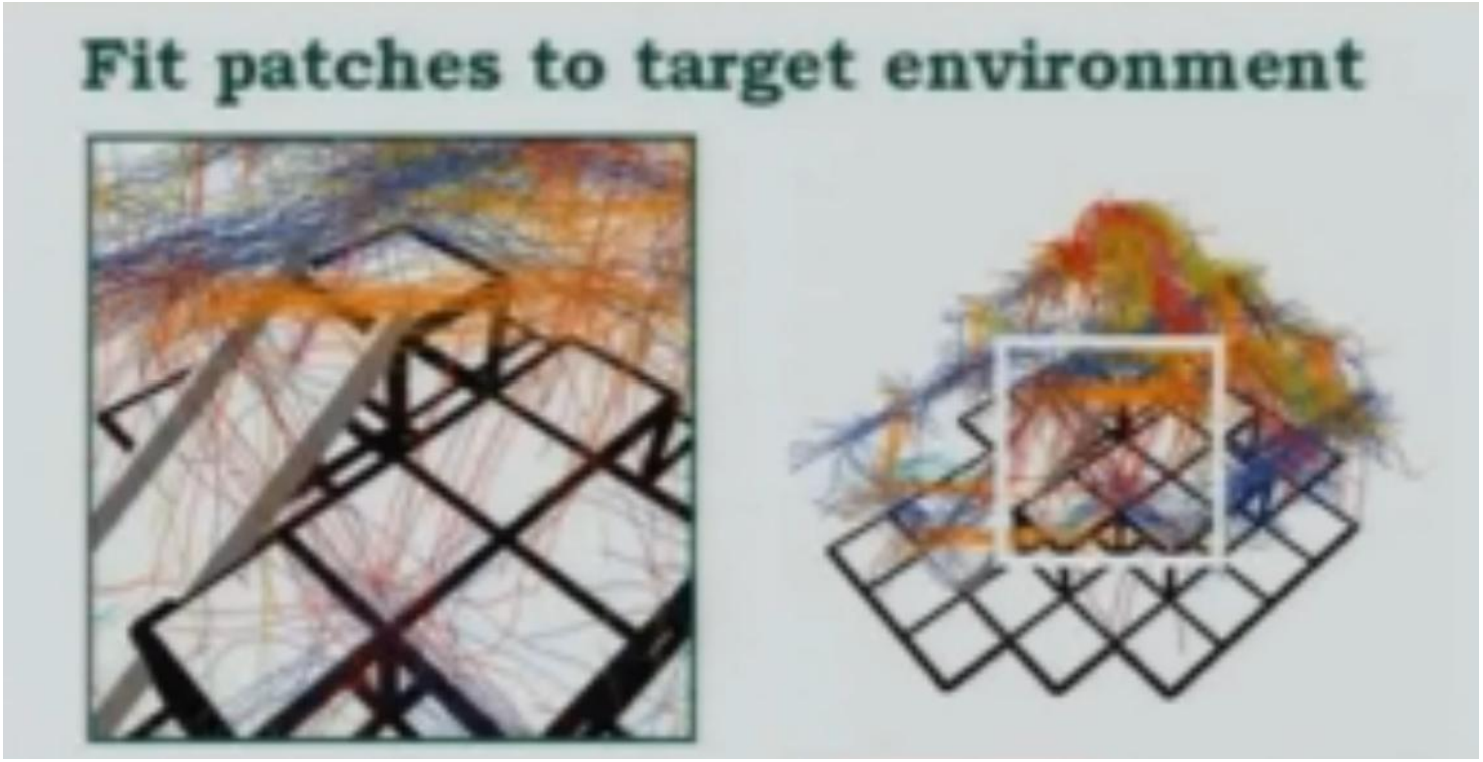
- Given a motion capture data where the human is interacting with the environment,
- Identify the motion patches using the body motion in two adjacent floor panels

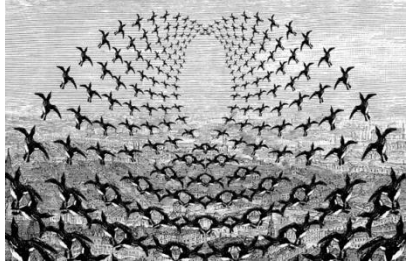




Motion Synthesis

Given a target environment, fit the patches into the environment by shape matching



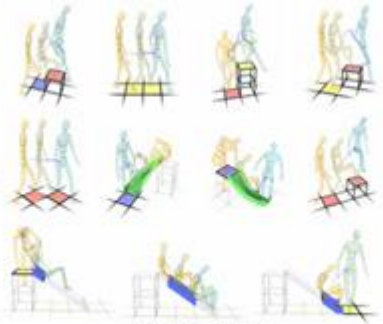


Motion Patches

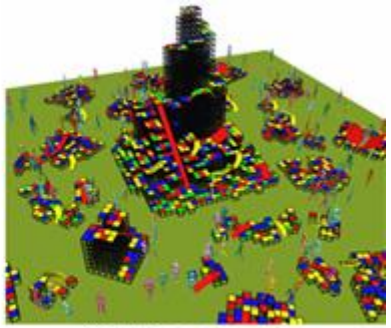
- Building Blocks for Virtual Environments
  - Embed the motions into the environment
  - The patches are spatially aligned



Motion capture
from the source environment

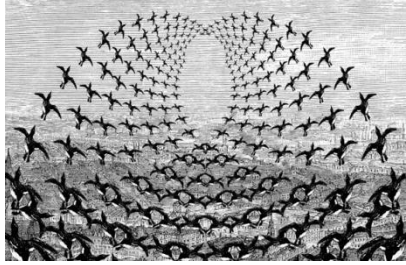


Building blocks
annotated with human motion data



Animation and control
in the target environment





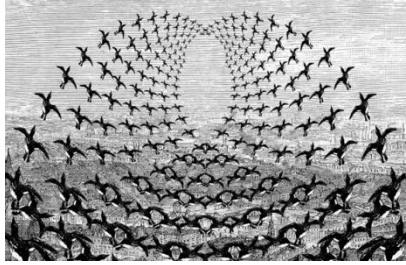
Motion Patches:

Building Blocks for Virtual Environments Annotated with Motion Data

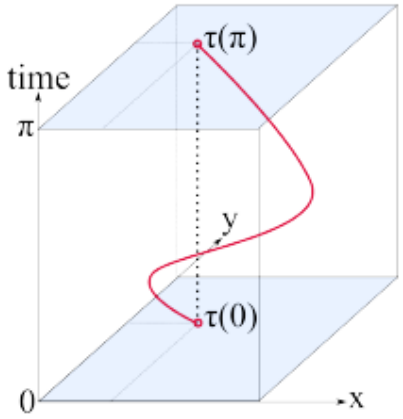
Kang Hoon Lee
Myung Geol Choi
Jehee Lee

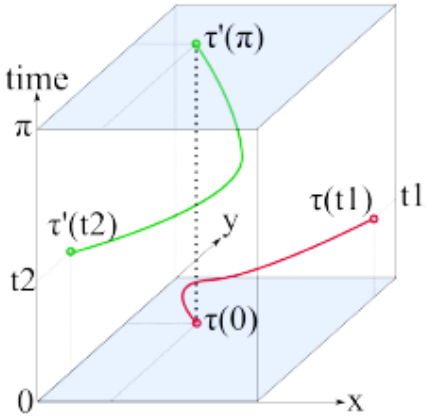
Seoul National University

Crowd Patches

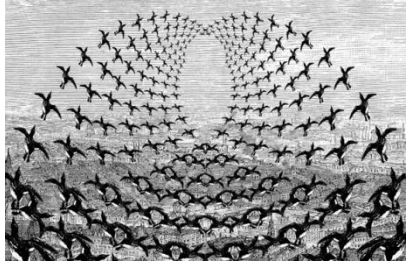


- A patch-based approach to generate scenes of crowded environment
- Crowds avoiding each other in the patches
- The timing and location the characters entering the patches are fixed so that the people can keep on entering / exiting
- The characters can be switched to make more variations







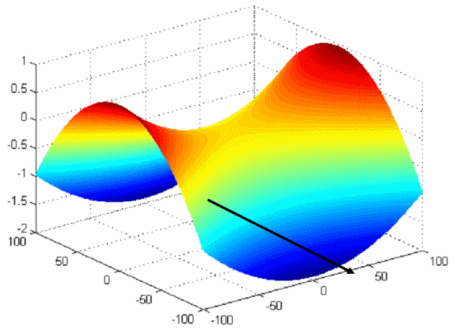


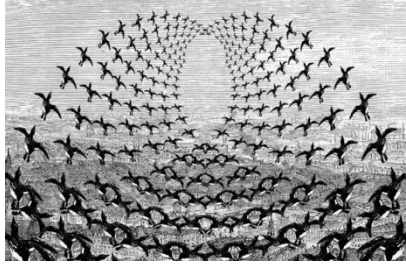
Overview

- Classic Agent-based Methods
- Flocking
  - Intention generator
  - Autonomous pedestrians
- Patch-based methods
  - Create scenes by building blocks
- **Collision avoidance**
  - Mathematical models
  - Global methods (Continuum crowds)



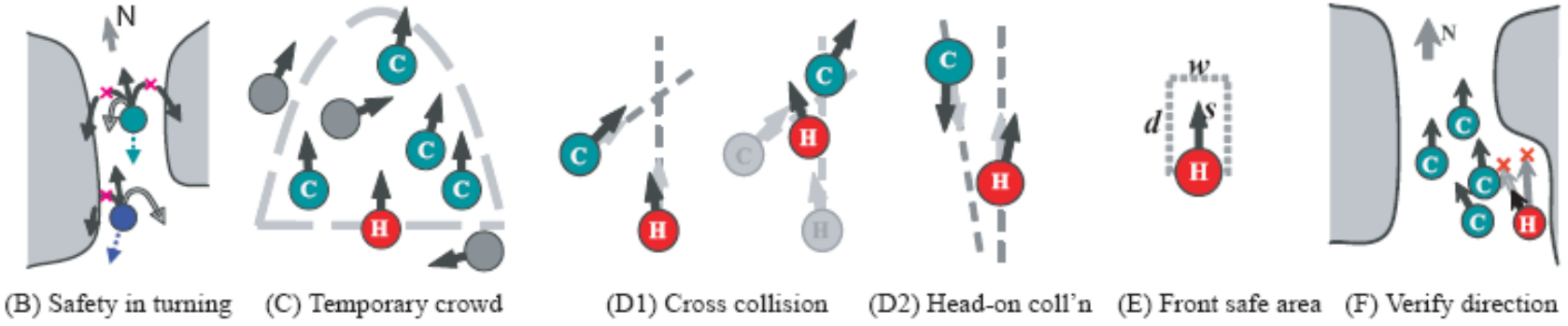






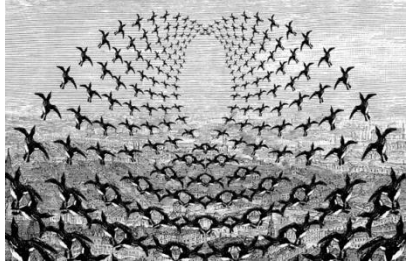
Collision avoidance

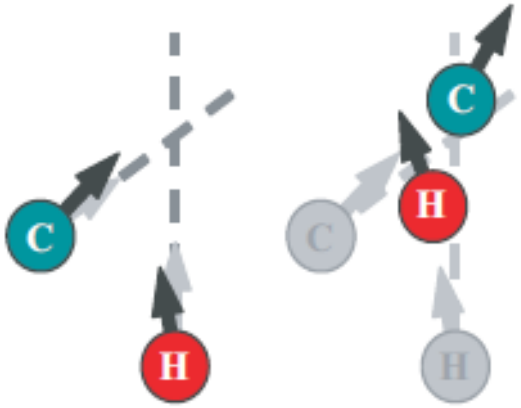
- Humans avoid others in streets in a complex way
- Sometimes wait, sometimes, move aside while walking
- Need to either
  - Model them based on a complex collision avoidance engine
  - Find a good mathematical model



(B) Safety in turning (C) Temporary crowd (D1) Cross collision (D2) Head-on coll'n (E) Front safe area (F) Verify direction

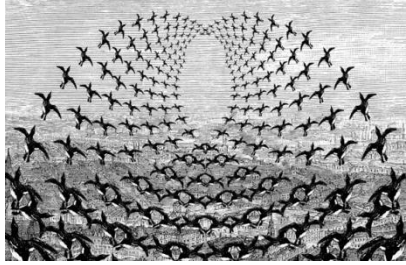
Avoid oncoming pedestrians





- If a cross collision is estimated by H with C
  - If H is arriving slightly earlier, it accelerates and turn away from C
  - If H is arriving slightly later, it decelerates and turn towards C
  - So C does the same

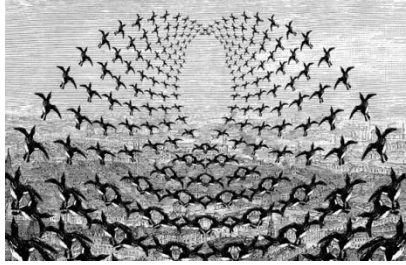
Head-on collision

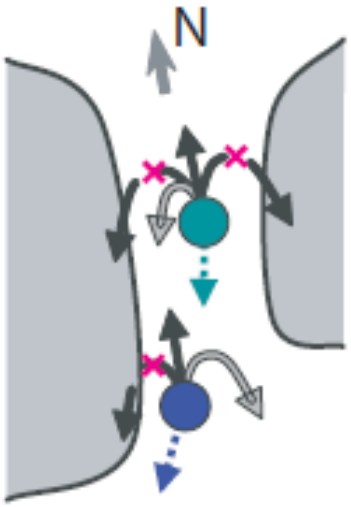




- If a head-on collision is estimated, the agents turn away from each other

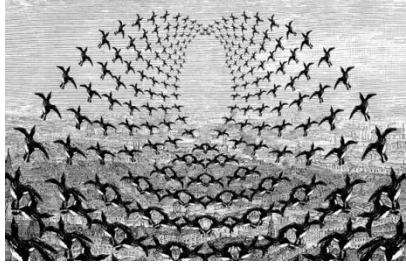
Safe turning



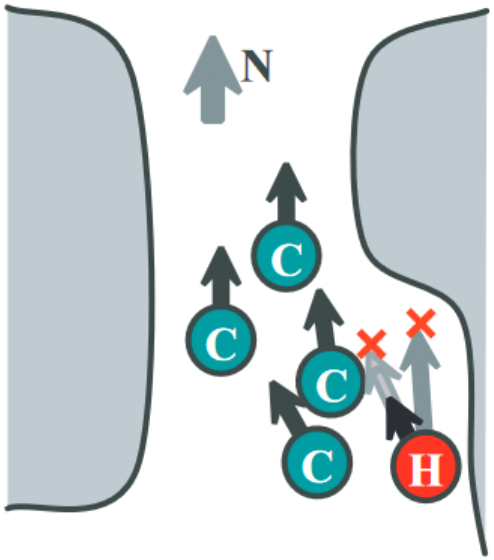


- If an agent needs to make a quick turn, the curvature of the turn is gradually increased until a collision free turn is found
- The velocity is decreased according to the curvature

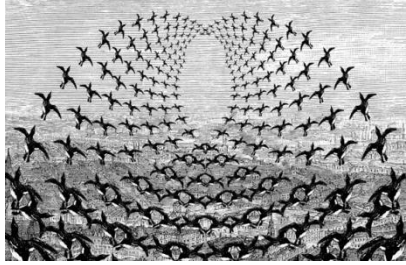
Verify directions



Confronted by static and dynamic threats, H picks obstacle-free direction (light gray arrow) and slows down (black arrow) to let others pass before proceeding

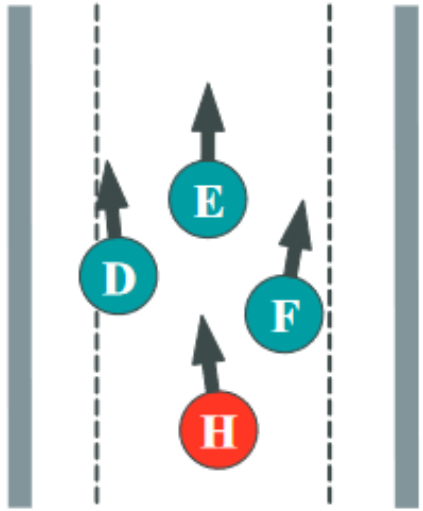


Passageway navigation (1)

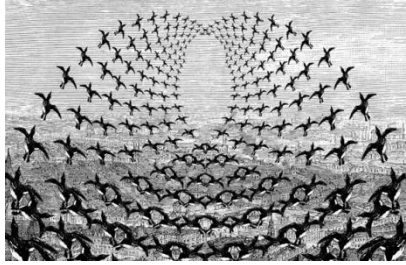


When everybody moving in the same direction, people spreads out

A repulsive force is added to each character to make this happen

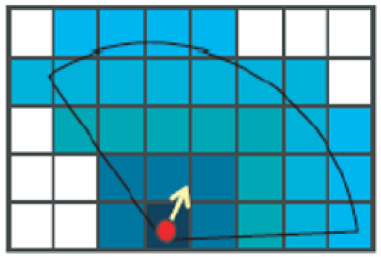


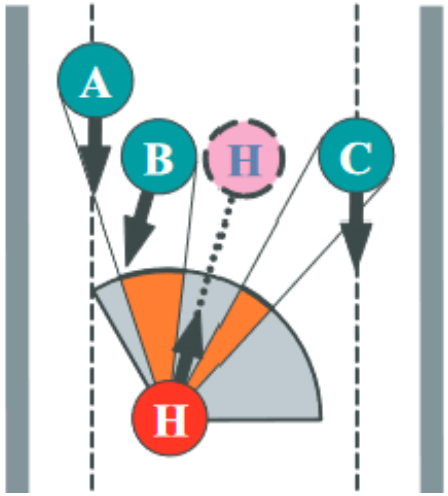
Passageway navigation (2)



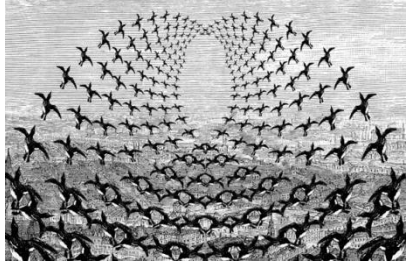
If H detects that its current direction is blocked by oncoming pedestrians, it will search within the safety fan for a safe interval to get through.

The search starts from H's current direction and continues clockwise.



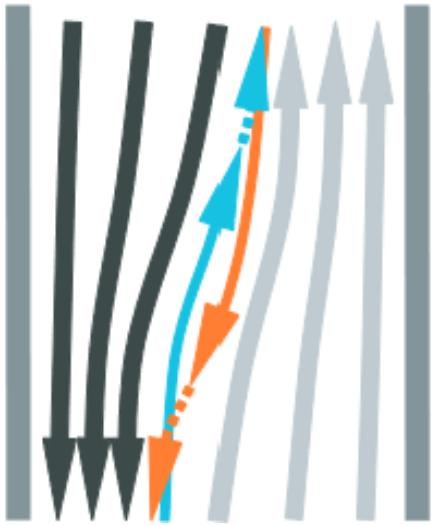


Passageway navigation (3)

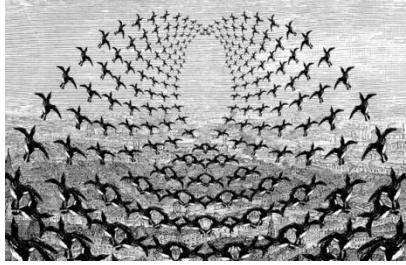


If the search succeeds, H will move in the safe direction found.

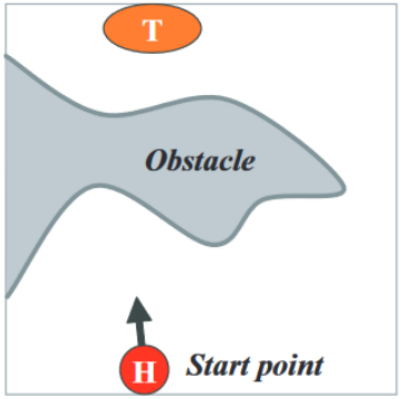
Otherwise, H will slow down and proceed in the rightmost direction within the safety fan.

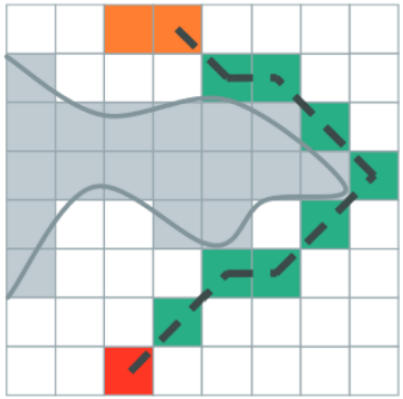


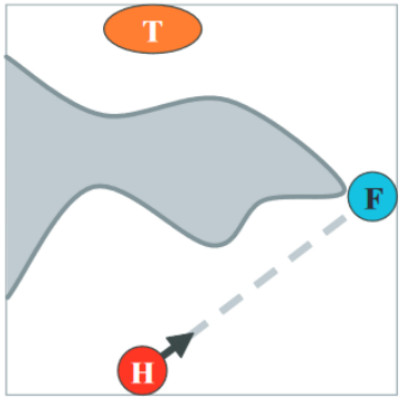
Navigation among static obstacles

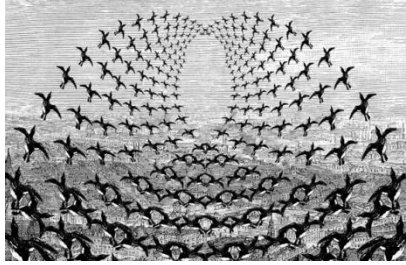


Given a discrete path selected by the A* search, pick the farthest visible point along the path and proceed toward it



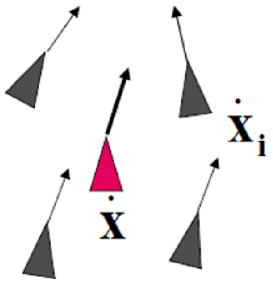


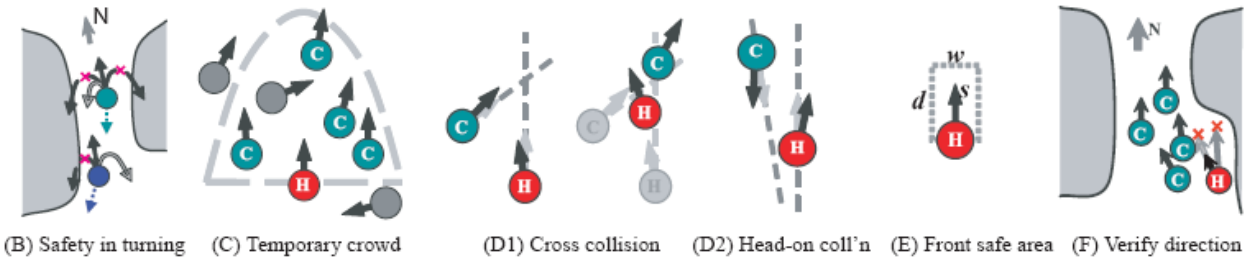




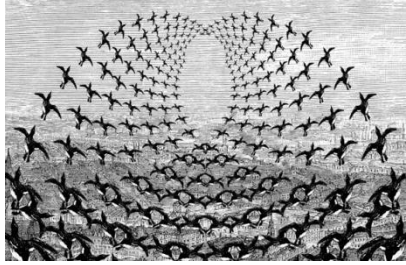
Issues

- Social forces can produce unpredictable forces
- Hand-tuned controllers blend between different controllers, which can result in discontinuities and possibly unpredicted motions
- This give rise to mathematical models





A Mathematical Model

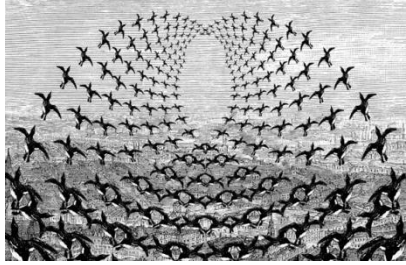


The behavior of the motions are explicitly defined by math

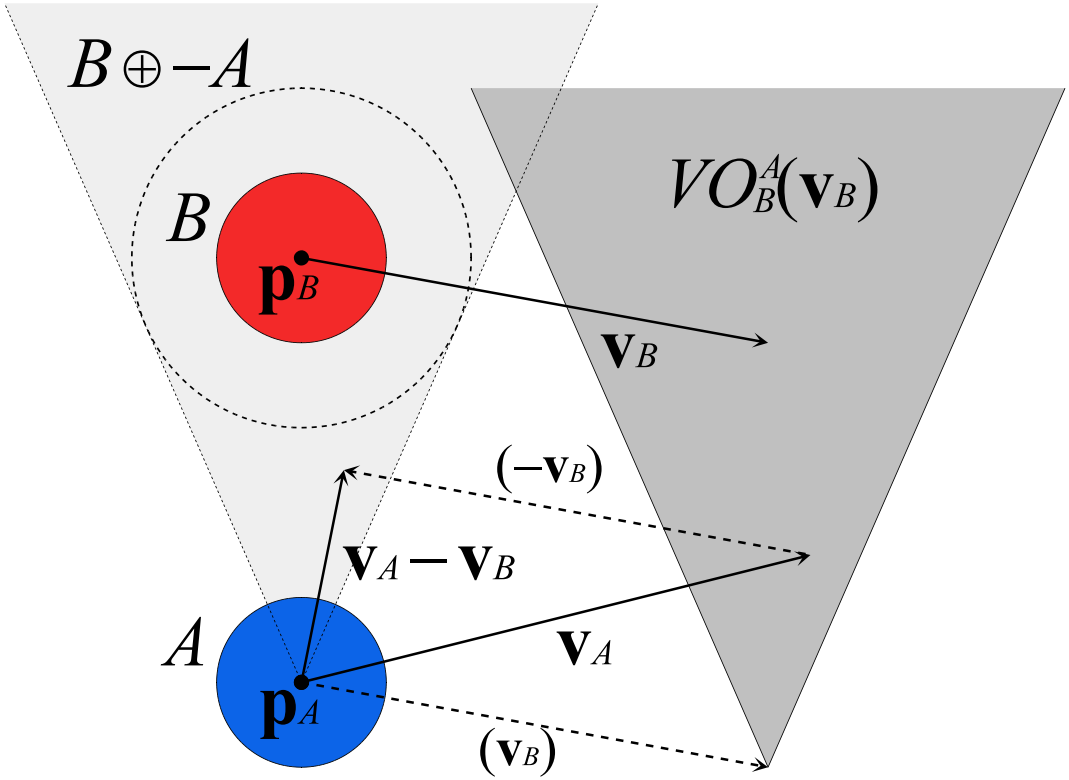
The characteristics of the controller are known in advance

Thus, resulting motions are more predictable

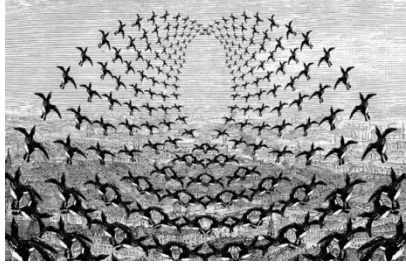
A Mathematical Model: Velocity Obstacle



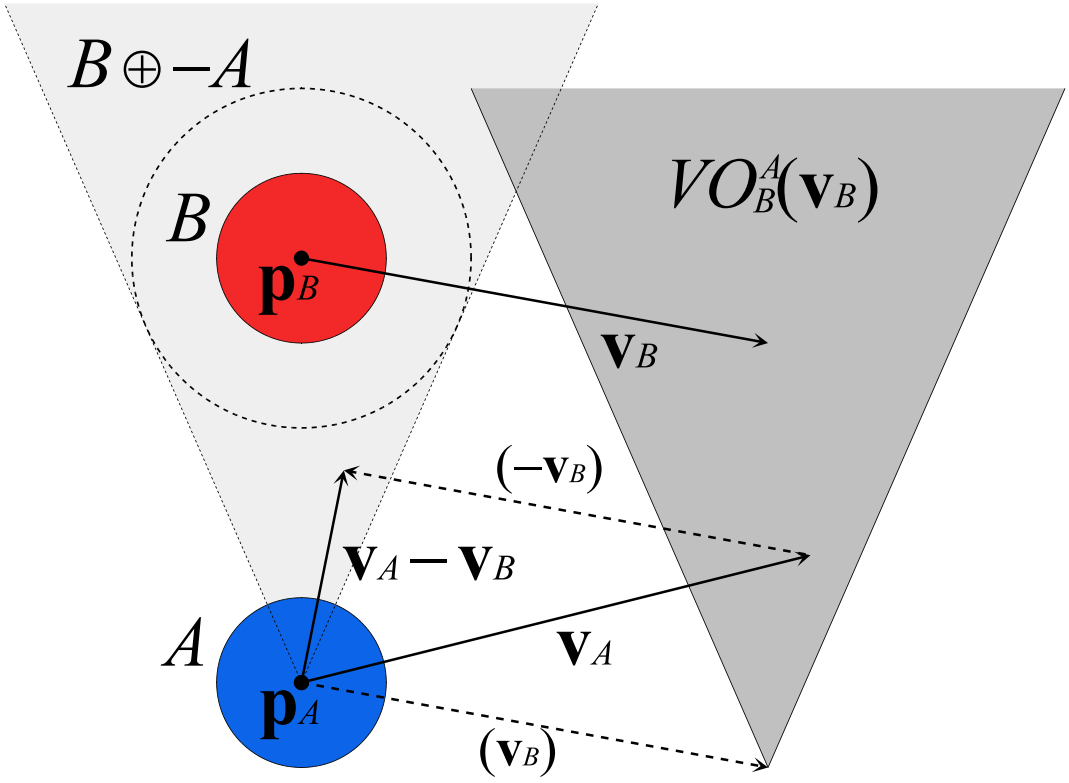
The set of all velocities of an agent that will result in a collision, assuming the other maintains its current velocity



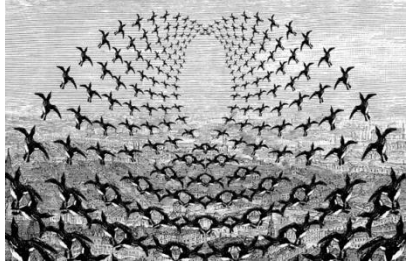
Velocity Obstacle (2)



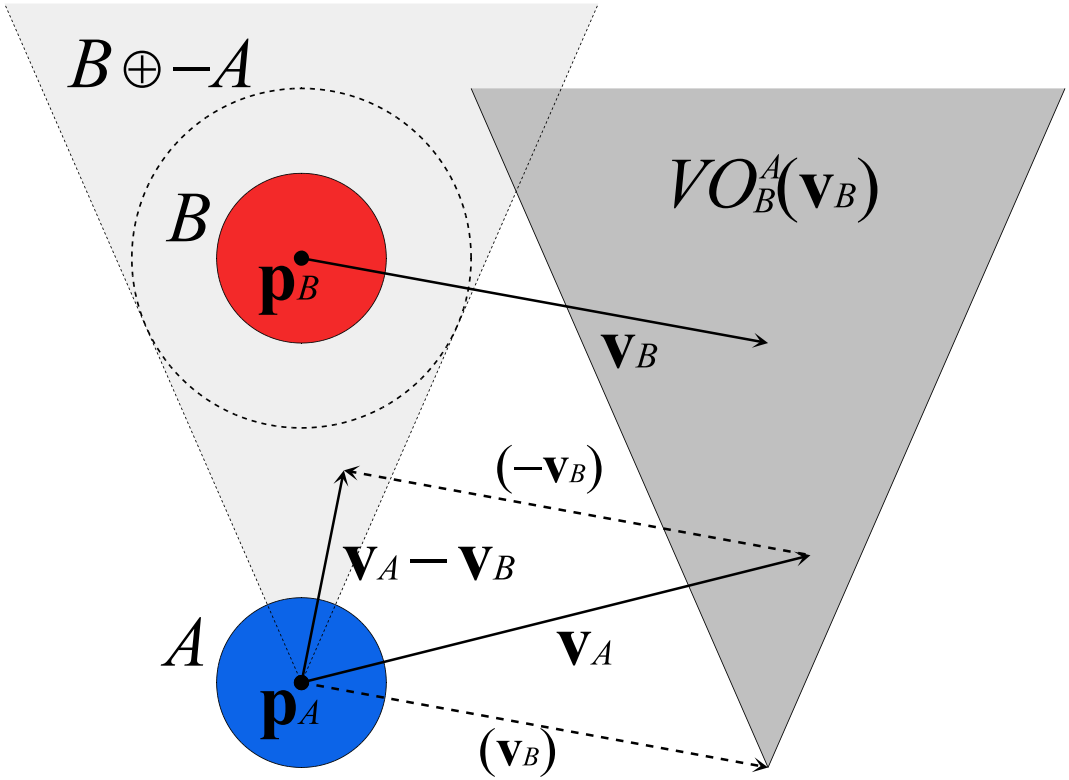
Strategy: Select a velocity that is closest to the desired velocity but outside the VO



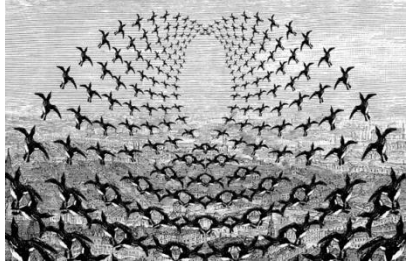
Velocity Obstacle (3)

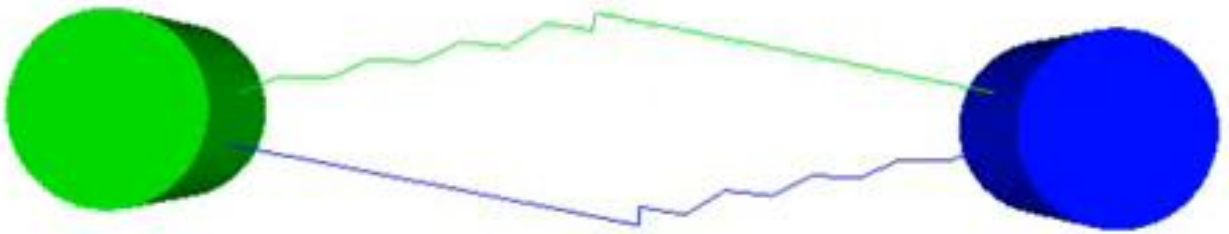


Good for collision avoidance, but can result in oscillation when both agents use the same strategy

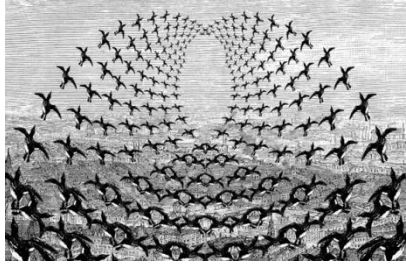


http://youtu.be/BNTNpYTw3_s



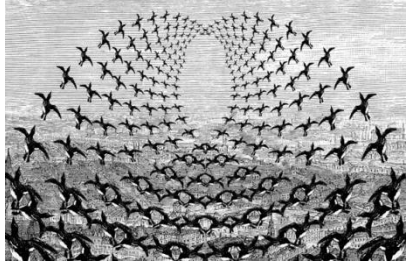


Reciprocal Velocity Obstacle (RVO)

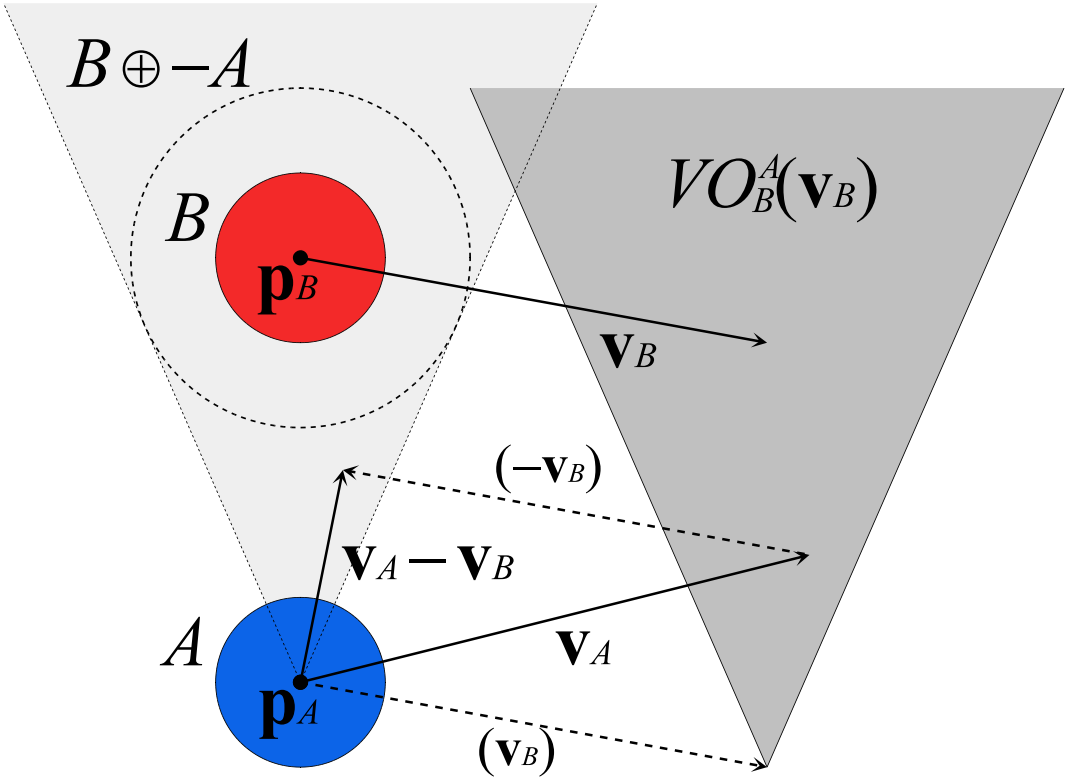


Idea: instead of choosing a new velocity outside the velocity obstacle, take the *average* of a velocity outside the velocity obstacle and the current velocity

Reciprocal Velocity Obstacle (RVO)

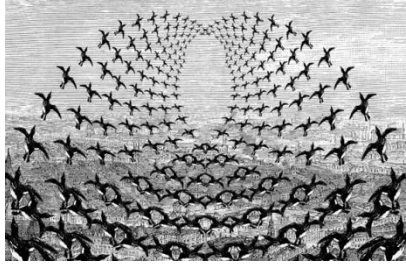


Idea: instead of choosing a new velocity outside the velocity obstacle, take the *average* of a velocity outside the velocity obstacle and the current velocity



http://youtu.be/9O-YkaiBVXw

Reciprocal Velocity Obstacle (RVO)



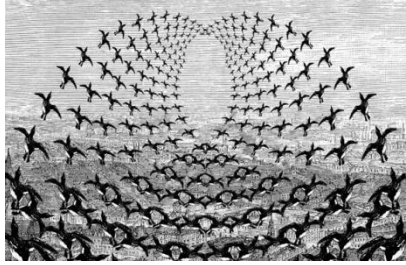
Idea: instead of choosing a new velocity outside the velocity obstacle, take the *average* of a velocity outside the velocity obstacle and the current velocity

$$\mathbf{v'_A} = \frac{\mathbf{v_A} + \overline{\mathbf{VO^A_B}}}{2}$$
$$2\mathbf{v'_A} - \mathbf{v_A} = \overline{\mathbf{VO^A_B}}$$

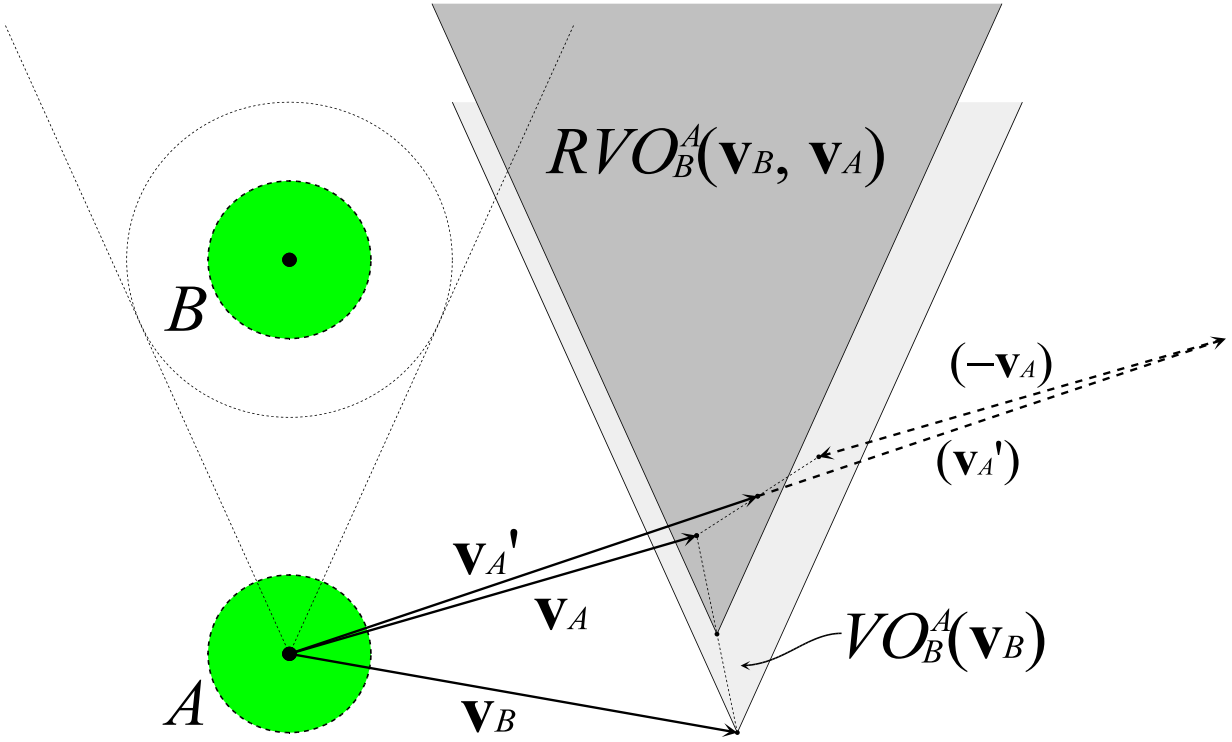
$\mathbf{v'_A}$is the set of points that its average with the velocity outside the VO
So, we avoid the to selecting the set of points their average is within the VO
The set of points $\mathbf{v'_A}$ falls inside VO are called RVO

$$RVO^A_B(v_B, v_A) = \{v'_A \mid 2v'_A - v_A \in VO^A_B(v_B)\}$$

Reciprocal Velocity Obstacle (RVO)

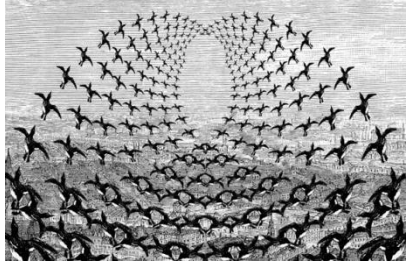


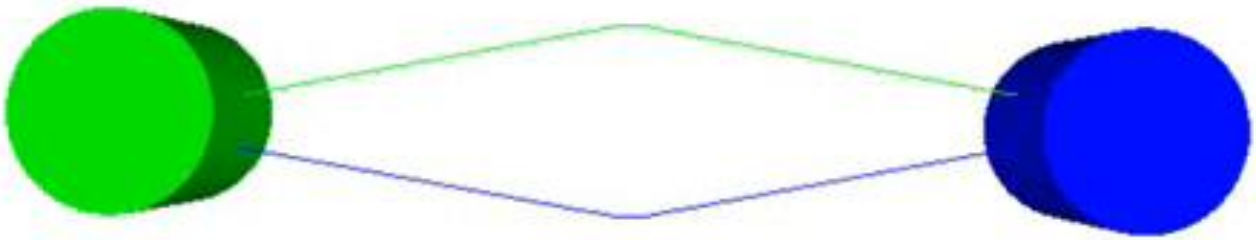
Idea: instead of choosing a new velocity outside the velocity obstacle, take the *average* of a velocity outside the velocity obstacle and the current velocity



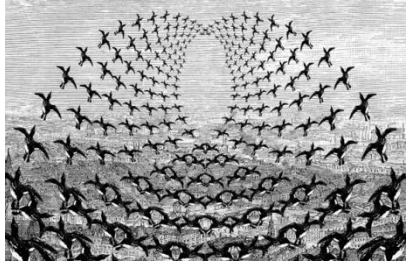
http://youtu.be/9O-YkaiBVXw

$$RVO^A_B(v_B, v_A) = \{v'_A \mid 2v'_A - v_A \in VO^A_B(v_B)\}$$





Reciprocal Velocity Obstacle (RVO) (2)

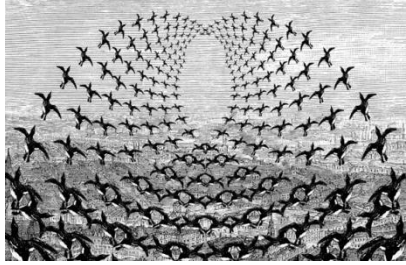


- No oscillation
- No global communication needed between the agents
- Simple and can handle multiple agents
  - Calculate the RVO with all the agents in the neighbours and select a velocity outside all the RVOs

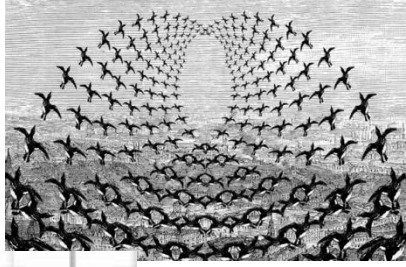
http://youtu.be/JX_GaFpIcqo

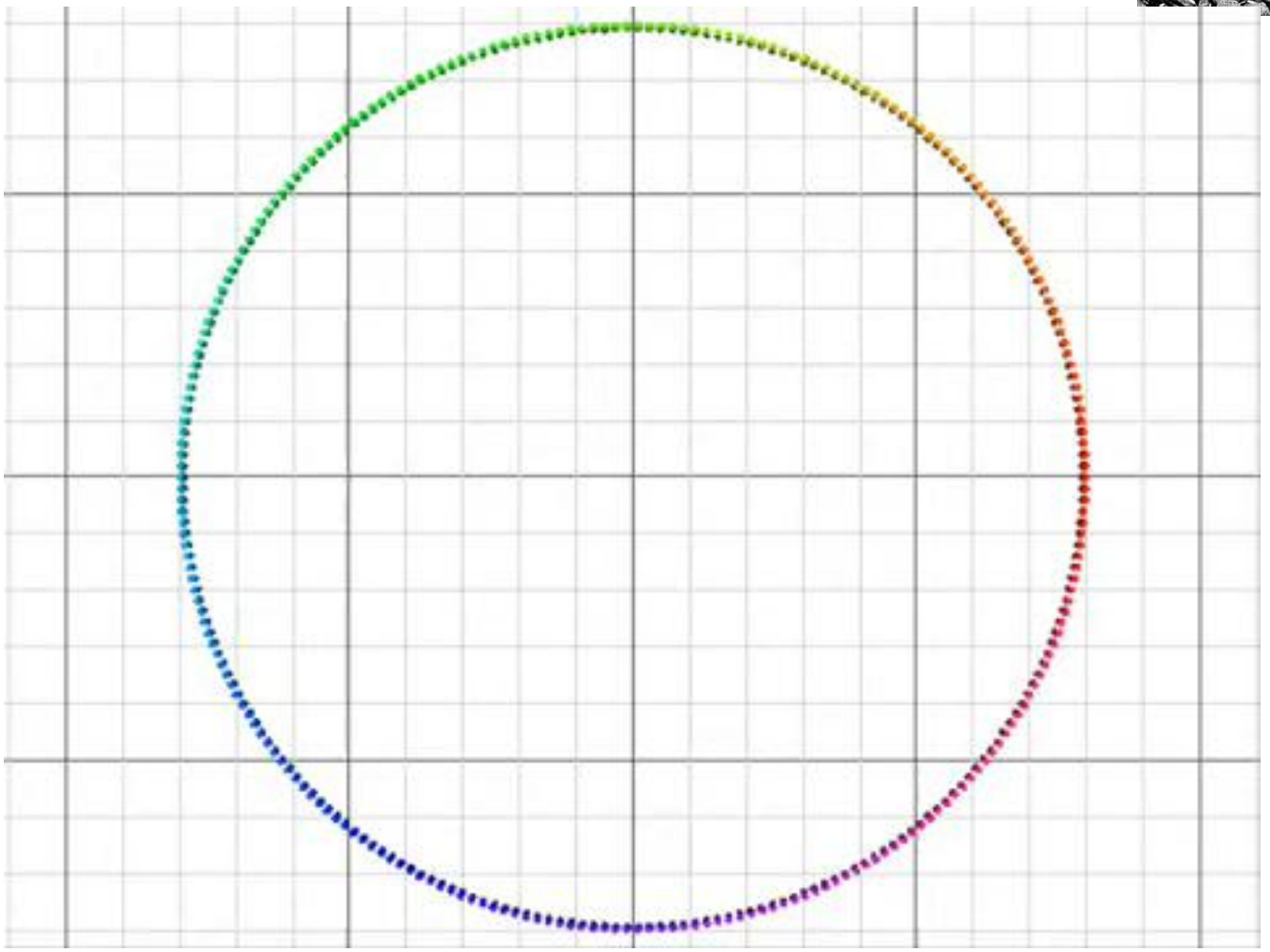
http://youtu.be/soHH-ocT1V8

http://youtu.be/nZ4mVCZRD0E

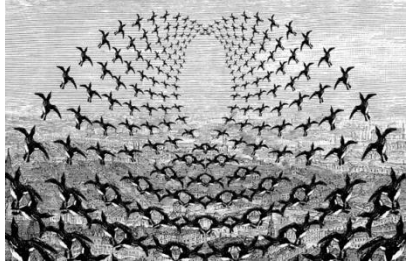








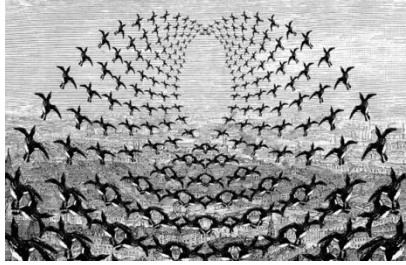
Collision Avoidance : Local vs. Global



- You may wish to plan a more advanced collision avoidance that can consider multiple obstacles
- The scalability of agent-based local controllers are linear – the computation will scale as the agents increase
  - Can we make it more efficient?

⟶ Global Model

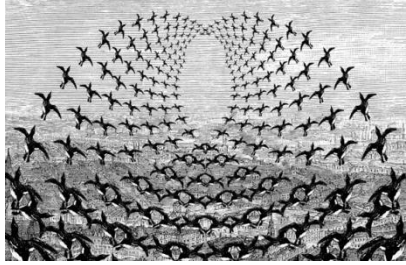
Continuum Crowds



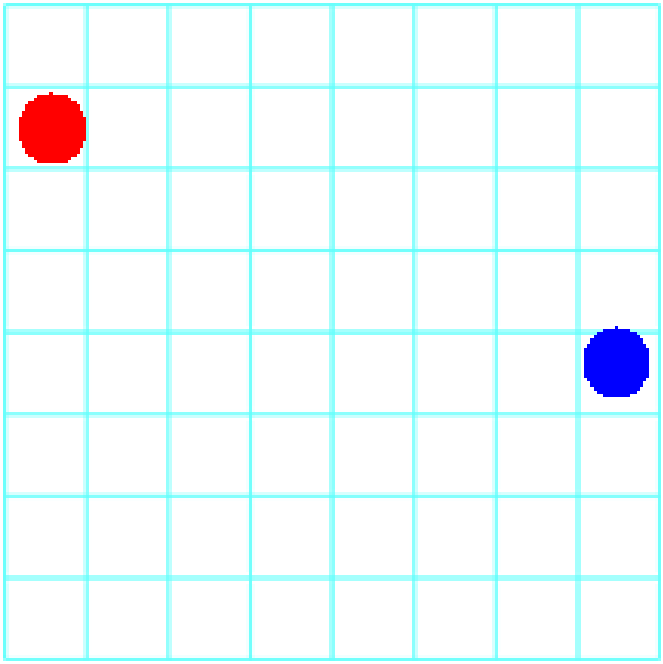
- Solving the path-planning and collision avoidance at the same time

1. Compute the potential field at every time step
   - Based on the other avatars and obstacles
   - The goal location
2. The character's movement determined based on the potential field
3. Update the potential field

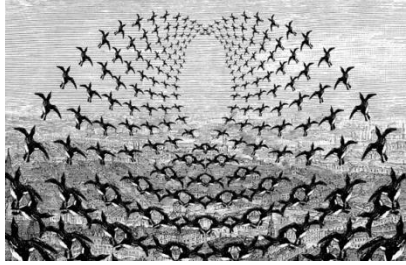
Continuum Crowds : procedure



- Discretize the space into grids
- Decide the start / goal of the characters



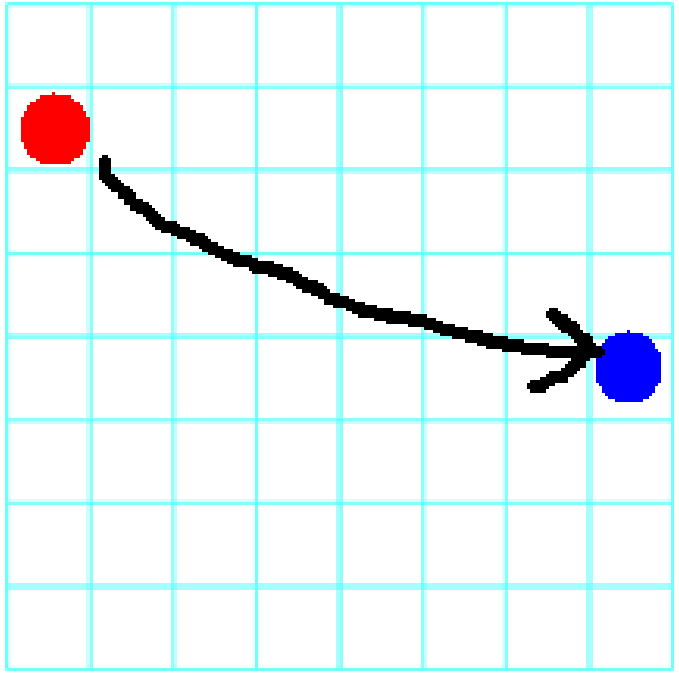
Continuum Crowds : procedure 2



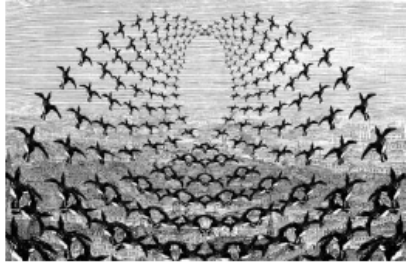
- The cost to the goal is going to be computed by the following function:

$$\underbrace{\alpha \int_P 1 ds}_{\text{Path Length}} + \underbrace{\beta \int_P \frac{1}{f} ds}_{\text{Time}} + \underbrace{\gamma \int_P \frac{g}{f} ds}_{\text{Discomfort}}$$

$$\int_P C ds, \quad \text{where} \quad C \equiv \frac{\alpha f + \beta + \gamma g}{f}$$



Continuum Crowds : procedure 2



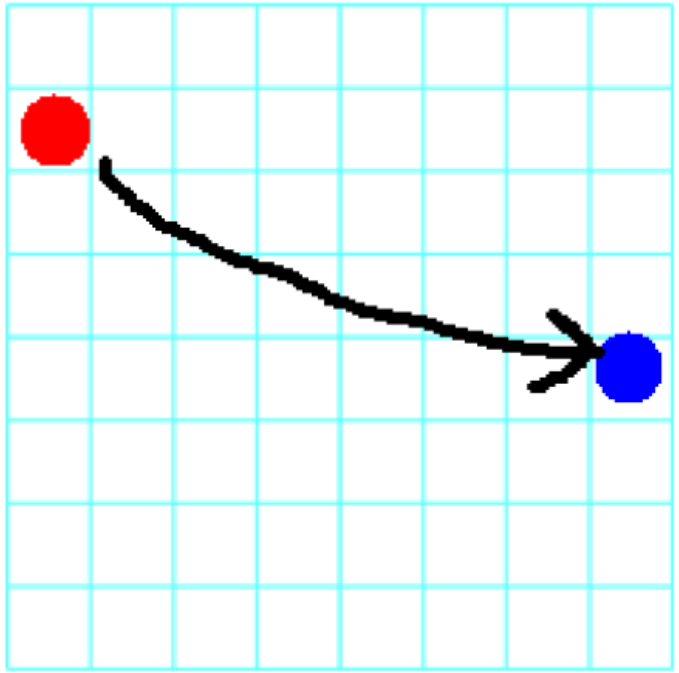
- The cost to the goal is going to be computed by the following function:

$$\alpha \int_P 1\,ds + \beta \int_P \frac{1}{f}\,ds + \gamma \int_P \frac{g}{f}\,ds$$

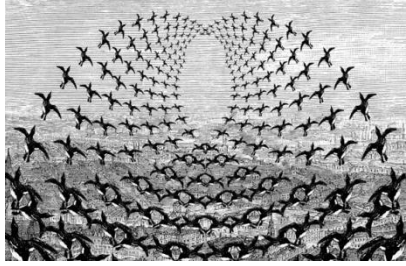
speed

Path Length | Time | Discomfort

$$\int_P C\,ds, \quad \text{where} \quad C \equiv \frac{\alpha f + \beta + \gamma g}{f}$$

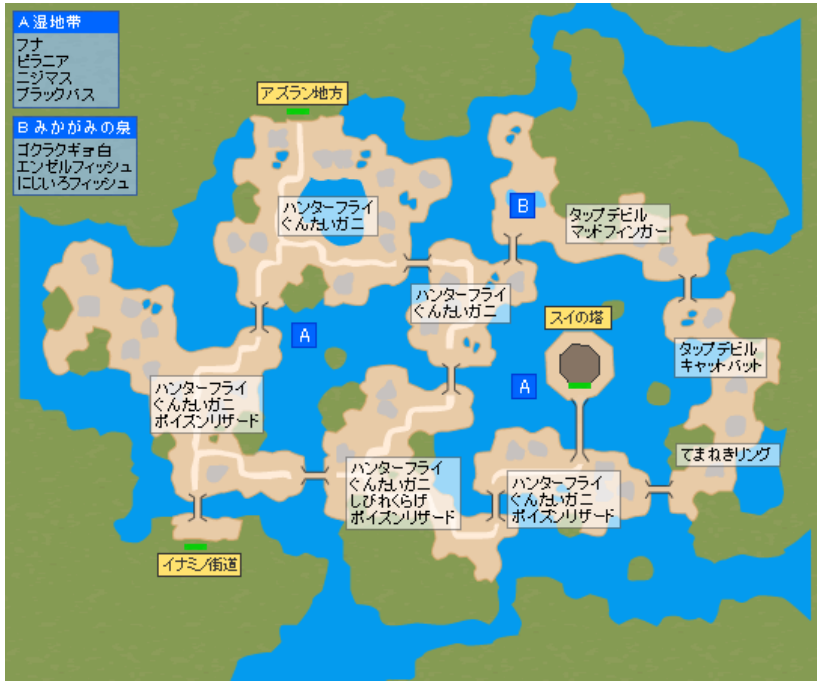


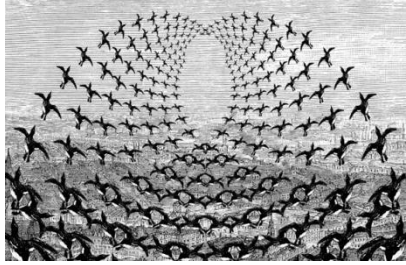
Why speed?



People walk at different speed at different kind of places

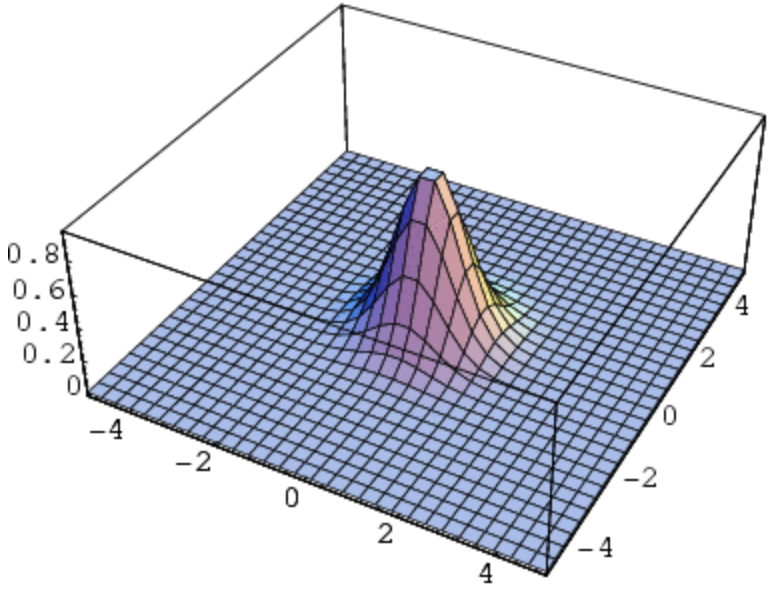
- Slow at muddy areas, mountains
- Zero at lake area
- Fast at ground

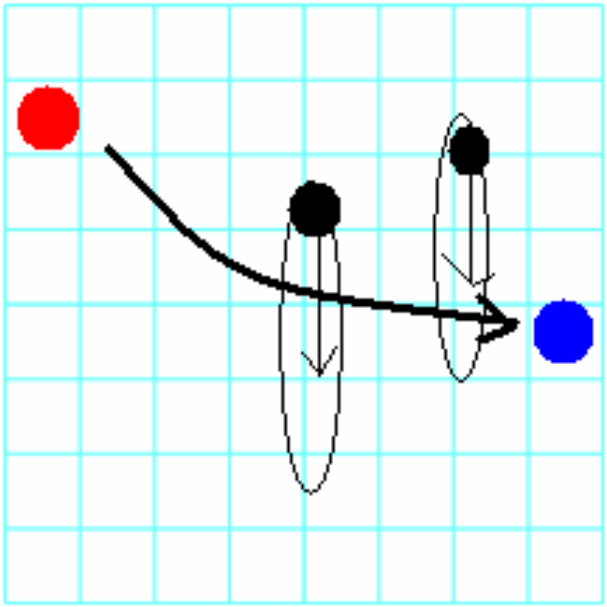




Discomfort Field

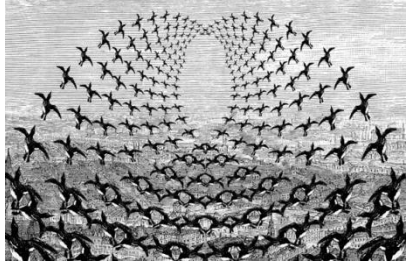
- Produced by other obstacles / characters

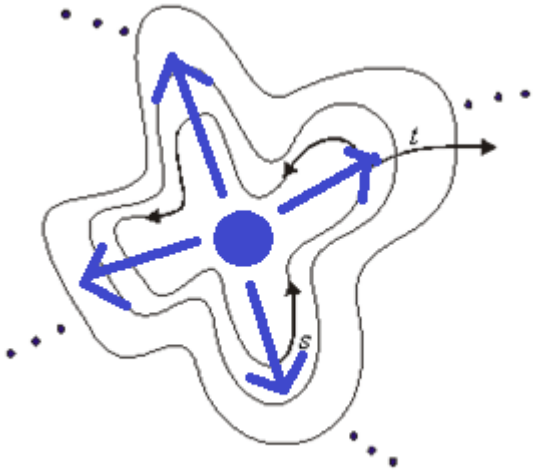




$$\underbrace{\alpha \int_P 1\,ds}_{\text{Path Length}} + \underbrace{\beta \int_P \frac{1}{f}\,ds}_{\text{Time}} + \underbrace{\gamma \int_P \frac{g}{f}\,ds}_{\text{Discomfort}}$$

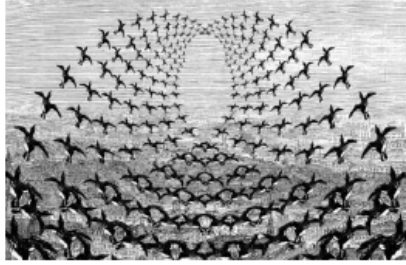
Continuum Crowds : procedure 2





- Starting from the goal we expand outwards and accumulate the cost *C*
  - *Fast Marching Method*
  - Similar to Djikstra's algorithm

Fast Marching Method

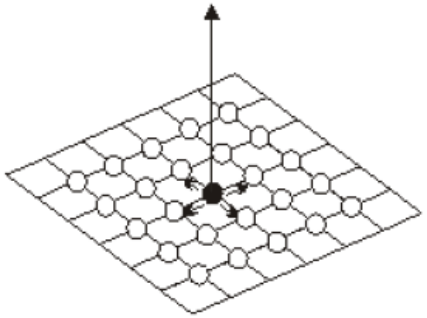


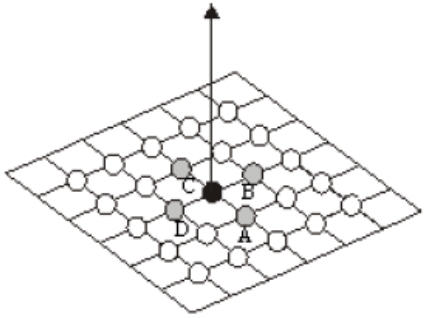
Initialize Step : Put the goal in the Alive Points

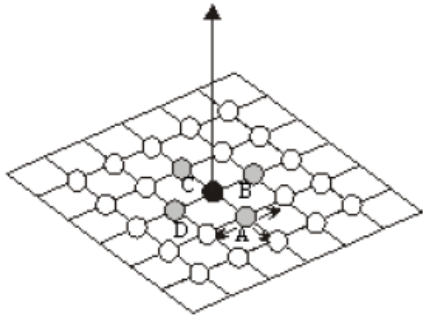
Alive Points (*A*): The set of points we know the cost ($T_{i,j}$)

Narrow Band: grid points neighbors of A.

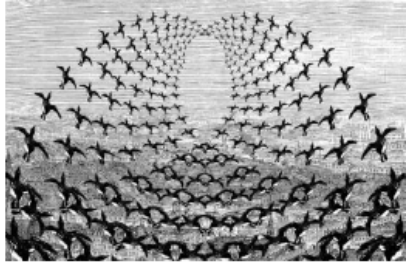
Far Away Points: all others grid points {i,j}. Set the cost ($T_{i,j}$) ∞ for all points in FarAway.





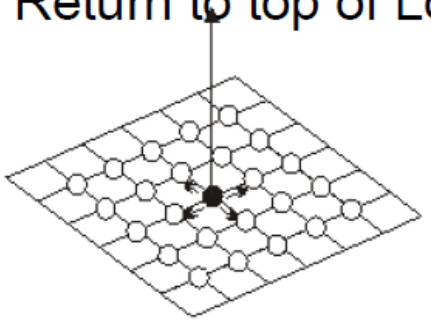


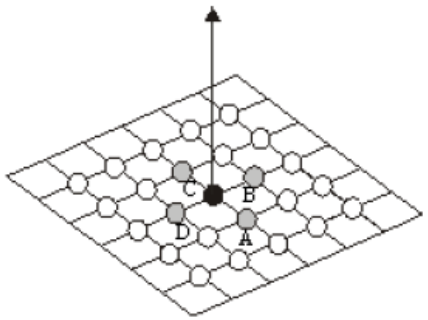
Marching Forwards

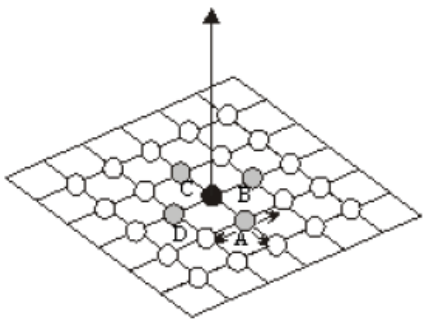


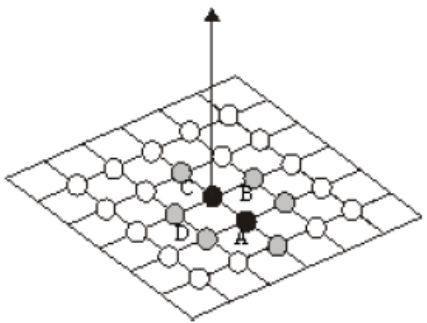
1. Begin Loop: Let (i_{min}, j_{min}) be the point in NarrowBand with the smallest value for $\emptyset$
2. Add the point (i_{min}, j_{min}) to A; remove it from NarrowBand.
3. Tag as neighbors any points $(i_{min}-1, j_{min})$, $(i_{min}+1, j_{min})$, $(i_{min}, j_{min}-1)$, $(i_{min}, j_{min}+1)$ that are either in NarrowBand or FarAway. If the neighbor is in FarAway, remove it from that list and add it to the set NarrowBand.
4. Recompute the values of $\emptyset$ at all neighbors

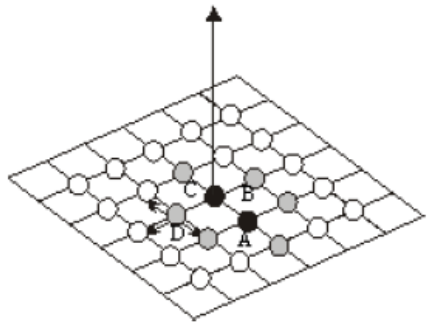
Return to top of Loop.

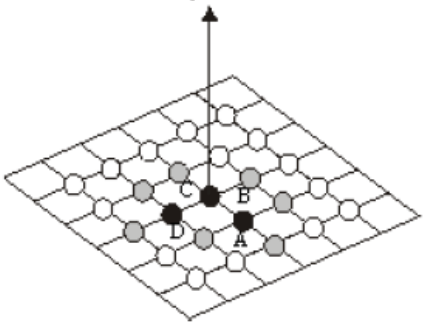




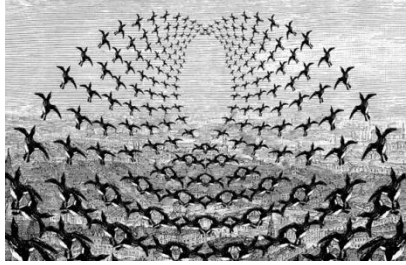








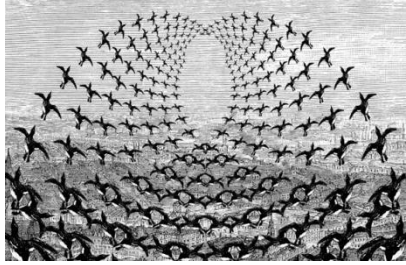
Potential Field

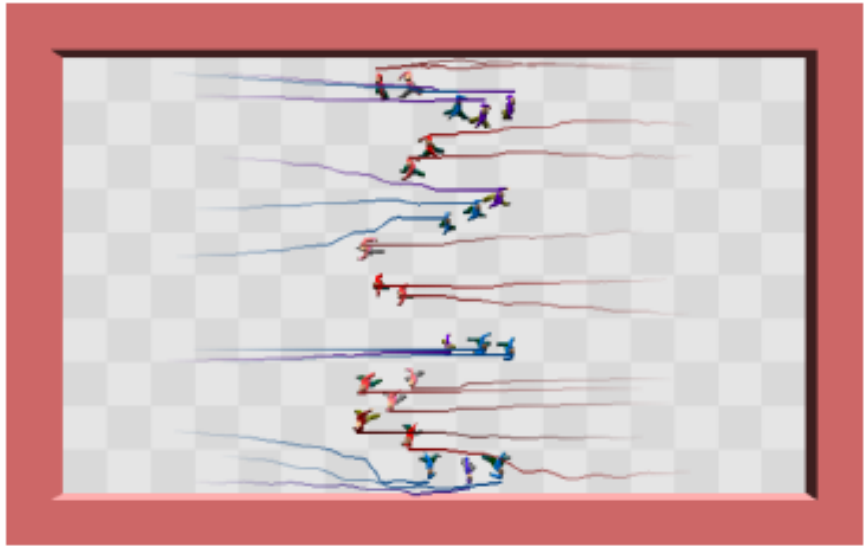


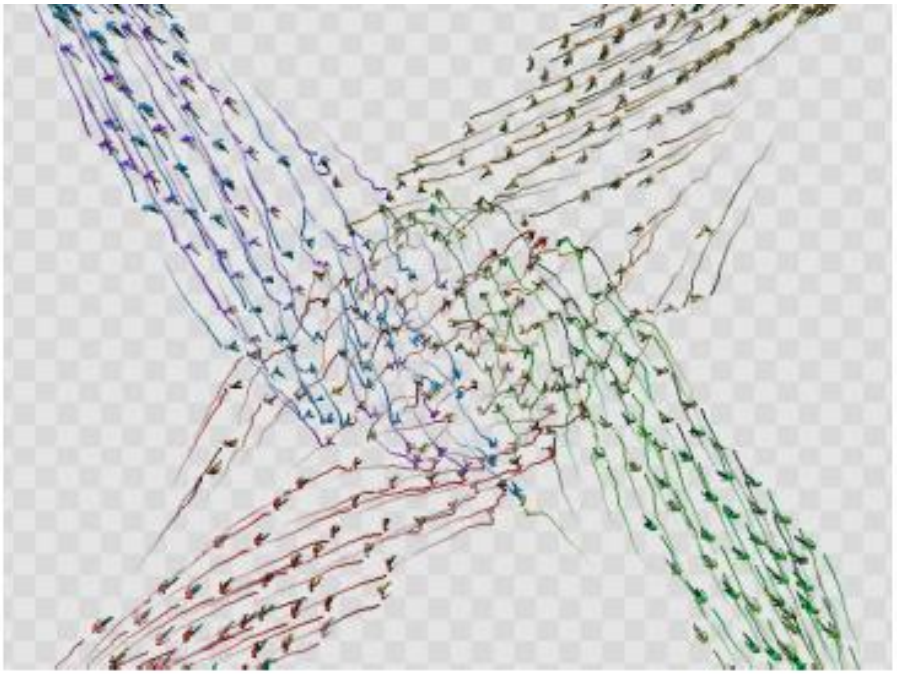


- If the group of people share the same speed, goal, and discomfort, we can use the same potential field for all these characters
  - Very efficient if there are little number of groups

Can simulate the dynamics of crowds

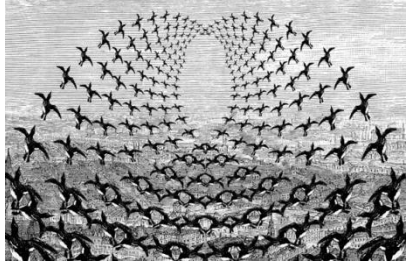






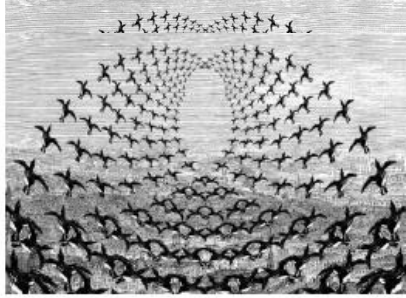
- A global approach (optimal towards the goal)
- Can simulate phenomena observed in real humans

http://www.youtube.com/watch?v=IqIuVhDFSp8



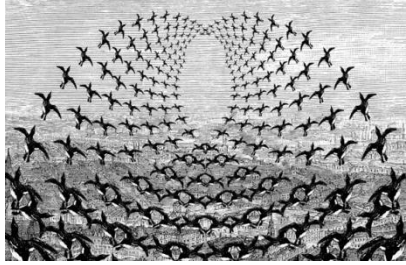
Comparisons

Summary and Discussions

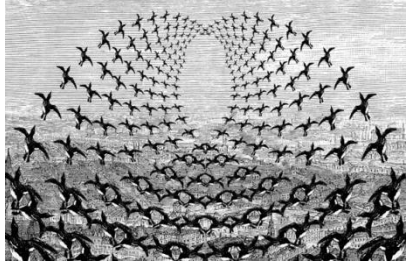


- Two streams
  - Agent-based
  - Global methods
- Applications
  - Animation, games
  - Evacuation analysis
- Collision avoidance is important not only for character animation but also for robotics

Reference



- REYNOLDS, C. W. 1987. Flocks, herds, and schools: A distributed behavioral model. In Computer Graphics (Proceedings of SIGGRAPH 87)
- TU, X., AND TERZOPOULOS, D. 1994. Artificial fishes: Physics, locomotion, perception, behavior. In Proceedings of SIGGRAPH 94, Computer Graphics Proceedings, Annual Conference Series, 43–50.
- SHAO, W., AND TERZOPOULOS, D. 2005. Autonomous pedestrians. In SCA '05: Proceedings of the 2005 ACM SIGGRAPH/Eurographics symposium on Computer animation, ACM Press, New York, NY, USA, 19–28.
- **Crowd Patches: Populating Large-Scale Virtual Environments for Real-Time Applications**.
  Barbara Yersin, Jonathan Maïm, Julien Pettré and Daniel Thalmann.
  ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games 2009.
- Kang Hoon Lee, Myung Geol Choi and Jehee Lee, Motion Patches: Building Blocks for Virtual Environments Annotated with Motion Data, accepted to ACM SIGGRAPH 2006.
- P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," Int. Journal of Robotics Research, vol. 17, no. 7, pp. 760–772, 1998.
- Jur van den Berg, Ming C. Lin, Dinesh Manocha "Reciprocal Velocity Obstacles for Real-Time Multi-Agent Navigation" Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2008.
- Adrien Treuille, Seth Cooper, Zoran Popovic, Continuum Crowds, SIGGRAPH 2005



ON THE IMPLEMENTATION OF FAST MARCHING METHODS

http://www2.imm.dtu.dk/pubdb/edoc/imm841.pdf